ABHISHEK THOMAS

Promotion 2024-2026

Active GLiNER: A Framework for Efficient NER Using LoRA Fine Tuning, Active Learning, and Synthetic Label Mixing

Master of Science in Data Science & Artificial Intelligence Strategy

Date of Defense: 05th December 2025
Name of Supervisor: Saeed Varasteh Yazdi

Acknowledgment

I would like to express my sincere gratitude to **Mr. Saeed Varasteh Yazdi**, Director of the Master in Data Science & Artificial Intelligence Strategy, for his exceptional guidance and support throughout the year. His leadership during the hackathons, coursework, and various academic activities significantly enriched my learning experience and contributed greatly to my academic and professional growth.

My deepest appreciation goes to **Mr. Philippe Fraisse**, Head of R&D at AI Factory Governance, for the invaluable mentorship he provided during this research. Thank you for giving me the opportunity to work under your guidance. Your expertise, innovative ideas, and the resources you made available particularly access to GPUs for conducting all experimental work on the GLiNER models were instrumental in bringing this dissertation to life.

I would also like to extend my thanks to **Dr. Cristina Oprean**, **Dr. Imène Brigui**, **Mr. Franck Jaotombo**, and all the faculty members for their continuous support, insightful guidance, and for creating an enriching learning environment throughout the program.

Finally, my deepest gratitude goes to my parents, **P. S. Thomas, Preethi Thomas** and my family members who have given me everything from day one. Their unconditional love, encouragement, and belief in me have been my greatest strength.

Table of Contents

Abstract

This thesis examines the current landscape of Named Entity Recognition (NER) research, with particular focus on the intersection of large language model capabilities and production deployment requirements. The thesis synthesizes recent advances in lightweight neural architectures, parameter efficient fine tuning methods, and synthetic data generation techniques that collectively enable the development of scalable NER systems. Key themes include the evolution of the GLiNER model family, the theoretical foundations and practical applications of Low Rank Adaptation (LoRA), and the emerging paradigm of knowledge distillation from large language models to specialized task models, multiple active learning strategies and we test multiple hypothesis using different active learning strategies and suggest best methods strategy for span based BERT models like GLiNER for NER. The analysis reveals significant opportunities for hybrid approaches that combine the performance advantages of state of the art language models with the efficiency characteristics required for enterprise deployment, while identifying critical gaps in current evaluation methodologies and practical implementation frameworks.

Chapter 1. Introduction

1.1 Goal of Master Thesis

Named Entity Recognition (NER) has emerged as a fundamental component in modern information extraction systems, serving as the backbone for numerous natural language processing applications ranging from document analysis to automated content understanding. However, the current landscape of NER solutions presents a significant paradox, while large language models (LLMs) demonstrate exceptional performance in entity recognition tasks, their deployment in production environments introduces substantial challenges including prohibitive computational costs, extensive inference times, and operational complexity that renders them impractical for many real world applications. [9]

This master thesis addresses the critical gap between state of the art NER performance and production feasibility by investigating a novel approach that combines the knowledge distillation capabilities of LLMs with lightweight, efficient neural architectures. The primary goal is to develop and evaluate a comprehensive framework that leverages GLiNER (Generalist Lightweight Model for Named Entity Recognition) as the foundation for creating production grade NER systems that can achieve near LLM performance while maintaining the efficiency and scalability required for enterprise deployment.

The research specifically focuses on the development of a modular, scalable system that utilizes Low Rank Adaptation (LoRA) fine tuning techniques to create task specific adapters from a combination of synthetic labels generated by LLMs and ground truth annotations provided by domain experts. This approach aims to dramatically reduce the labeling burden on businesses while maintaining high quality entity recognition performance across diverse domains and use cases.

Through rigorous experimentation and evaluation using the MIT Movies NER dataset [3] as a simulation environment, this thesis seeks to establish empirical evidence for the effectiveness of mixed synthetic and ground truth label training, quantify the performance trade offs between different mixing ratios, and demonstrate the practical viability of deploying lightweight NER models in production environments where cost efficiency and scalability are paramount considerations.

1.2 Introduction to the Problem

The evolution of information extraction systems has reached a critical juncture where the demand for accurate, fast, and cost effective named entity recognition significantly exceeds the capabilities of current deployment paradigms. Modern enterprises generate and process vast quantities of unstructured text data daily, requiring sophisticated NER capabilities to extract meaningful insights for business intelligence, regulatory compliance, and automated decision making processes.

Contemporary NER solutions predominantly rely on large language models such as GPT 4, Claude, or Gemini, which demonstrate remarkable performance across diverse entity types and domains. These models excel in few shot and zero shot scenarios, can handle complex entity relationships, and provide contextual understanding that surpasses traditional rule based or early neural approaches. However, their deployment in production environments introduces a cascade of operational challenges that fundamentally limit their practical applicability. [9]

The first and most significant challenge is computational cost. Large language models require substantial GPU resources for inference, with costs scaling linearly with the volume of processed text. For organizations processing millions of documents or requiring real time entity extraction, the financial implications become prohibitive. Additionally, these models often exceed the computational capacity available for on premises deployment, forcing organizations to rely on external API services that introduce additional latency, data privacy concerns, and dependency on third party infrastructure.

The second critical limitation involves the inflexibility of LLMs for task specific optimization. Most commercial LLM providers do not offer fine tuning capabilities, preventing organizations from adapting models to domain specific entity types or improving performance on specialized vocabularies. Even when fine tuning is available, the computational requirements for training large models make it accessible only to organizations with substantial technical infrastructure and financial resources.

Furthermore, LLMs suffer from inherent scalability constraints when deployed in enterprise environments. Token limits, rate limiting, retry mechanisms, and API quotas create bottlenecks that can severely impact system throughput. The unpredictable nature (non deterministic) of LLM response times and occasional API failures require sophisticated error handling and fallback mechanisms, adding complexity to system architecture and maintenance.

The unpredictable nature of LLM response times and occasional API failures require sophisticated error handling and fallback mechanisms, adding complexity to system

architecture and maintenance. Moreover, relying on LLMs introduces performance instability, companies frequently switch underlying model versions, sometimes substituting full models with quantized ones for cost or efficiency[1]. Although API providers advertise consistent service for specific use cases, users lack transparency regarding these internal model changes, which can subtly yet significantly affect performance and reliability. Additionally, the lack of complete visibility raises concerns about data privacy, as it is uncertain whether or how the input data is being collected or used for the provider's internal benefits.

These challenges have created a significant demand for alternative approaches that can bridge the performance gap between traditional lightweight NER models and state of the art LLMs while maintaining the operational efficiency required for production deployment. The ideal solution must combine the accuracy and flexibility of modern language models with the speed, cost effectiveness, and reliability characteristics essential for enterprise scale information extraction systems.

1.3 Overview of the Report

This thesis presents a comprehensive investigation into the development and evaluation of a production ready named entity recognition system that addresses the fundamental trade offs between performance, efficiency, and scalability in modern information extraction applications. The research is structured to provide both theoretical foundations and practical implementations, demonstrating the viability of hybrid approaches that combine the strengths of large language models with lightweight, deployable architectures.

The report begins with an extensive literature review that establishes the current state of named entity recognition research, tracing the evolution from rule based systems through statistical methods to contemporary neural architectures. This section provides detailed analysis of existing state of the art approaches, with particular emphasis on the GLiNER family of models and their architectural innovations that enable efficient entity recognition through bidirectional transformer architectures and single pass processing mechanisms.

The literature review further explores the theoretical foundations of parameter efficient fine tuning methods, specifically focusing on Low Rank Adaptation (LoRA) techniques that enable rapid adaptation of pretrained models to specific domains while maintaining computational efficiency. The discussion includes comprehensive coverage of synthetic data generation methodologies, evaluation frameworks for NER systems, and the practical considerations involved in manual annotation processes that inform the cost benefit analysis of different labeling strategies.

The methodology section details the experimental framework developed to evaluate the proposed approach, including the rationale for dataset selection, the design of the modular software architecture that follows SOLID principles for maximum adaptability, and the implementation of the five core functions that comprise the system, zero shot prediction, uncertainty based ranking, fine tuning with mixed labels, prediction with trained adapters, and comprehensive evaluation with confidence analysis.

The results section presents empirical findings from extensive experiments comparing different mixing ratios of synthetic and ground truth labels, analyzing the relationship between training data size and model performance, and quantifying the efficiency gains achieved through the proposed approach. Special attention is given to confidence based uncertainty sampling strategies and their effectiveness in identifying examples most suitable for LLM labeling.

Additional experimental investigations explore various aspects of the system including baseline comparisons between different model architectures, analysis of entity type coverage and distribution effects, and evaluation of the system's performance across different confidence thresholds and uncertainty measures.

The report concludes with a discussion of future research directions, including potential extensions to multilingual scenarios, integration with active learning frameworks, and scalability considerations for deployment in large scale production environments. The findings contribute to the broader understanding of knowledge distillation in NER applications and provide practical guidance for organizations seeking to implement efficient, scalable entity recognition systems without compromising on accuracy or reliability.

Chapter 2. Literature Review

2.1 NLP, Information Extraction, and Named Entity Recognition Fundamentals

Natural Language Processing has undergone a profound transformation over the past decade, evolving from rule based systems and statistical methods to sophisticated neural architectures capable of understanding and generating human language with unprecedented accuracy. Information extraction, a core subfield of NLP, focuses on automatically identifying and structuring relevant information from unstructured text sources, enabling machines to convert human readable documents into structured data suitable for computational analysis and decision making processes.

Named Entity Recognition represents one of the most fundamental and well studied tasks within information extraction, involving the identification and classification of named entities such as persons, organizations, locations, dates, and domain specific entities within text. The task encompasses two primary components: entity boundary detection, which determines the exact span of text corresponding to an entity, and entity classification, which assigns appropriate semantic labels to identified entity spans. The complexity of NER arises from the inherent ambiguity of natural language, where the same textual span may represent different entity types depending on context, and entity boundaries may be unclear due to compositional naming conventions or nested entity structures.

The significance of accurate NER extends far beyond academic research, forming the foundation for numerous downstream applications including information retrieval, question answering systems, knowledge graph construction, sentiment analysis, and automated document processing. In enterprise environments, NER systems enable automatic extraction of critical information from legal documents, financial reports, medical records, and customer communications, facilitating compliance monitoring, risk assessment, and business intelligence generation at scale.

Traditional approaches to NER relied heavily on hand crafted features and rule based systems that required extensive domain expertise and manual engineering effort. These systems typically employed regular expressions, gazetteer lookups, and linguistic pattern matching to identify potential entity mentions, often achieving reasonable performance within constrained domains but struggling to generalize across different text types or adapt to new entity categories without significant manual intervention.

The introduction of statistical machine learning methods marked a significant advancement in NER capabilities, with Conditional Random Fields (CRFs) and Hidden Markov Models (HMMs) becoming dominant approaches throughout the early 2000s. These methods leveraged sequence labeling formulations that could model dependencies between adjacent entity labels while incorporating contextual features such as part of speech tags, word shapes, and surrounding word contexts. However, these approaches still required extensive feature engineering and often struggled with complex entity types or ambiguous contexts.

2.2 History of Named Entity Recognition

The development of Named Entity Recognition can be traced through several distinct evolutionary phases, each characterized by fundamental shifts in underlying methodologies and technological capabilities. The earliest NER systems emerged in the 1990s as part of the Message Understanding Conference (MUC) evaluations[2], which established standardized evaluation frameworks and datasets that continue to influence modern research practices.

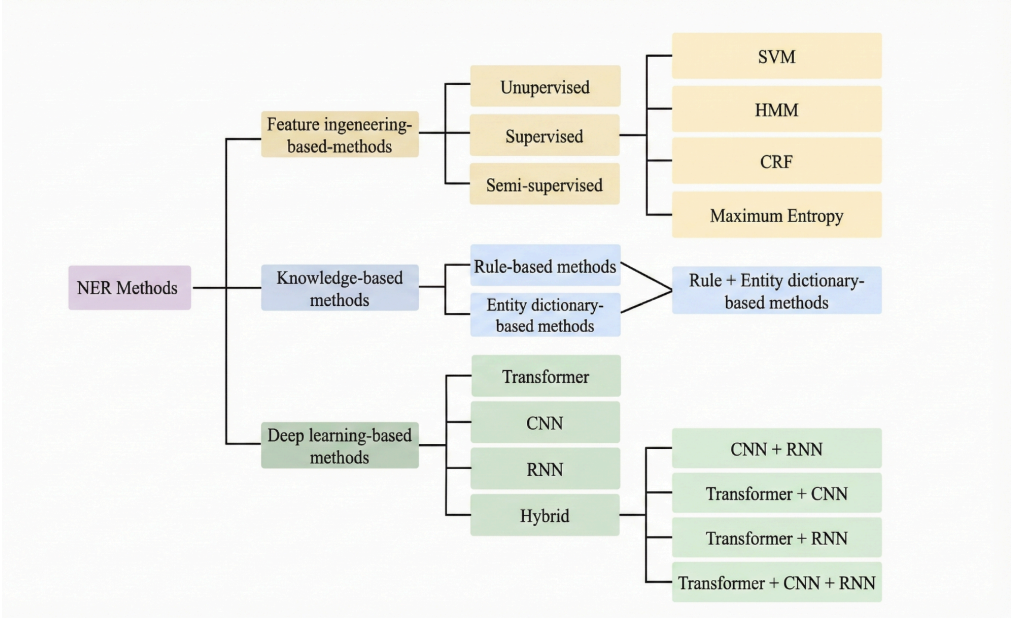

Fig 2.2.1 : History of NER [3]

The rule based era, spanning from the early 1990s through the early 2000s, was characterized by systems that relied on manually crafted patterns, dictionaries, and heuristics to identify entity mentions. These systems achieved remarkable precision within specific domains but suffered from limited recall and poor generalization capabilities. Notable examples include the FASTUS system[2] developed for information extraction from news articles and early biomedical NER systems that leveraged domain specific terminologies and naming conventions.

The statistical learning period, beginning in the late 1990s and continuing through the 2010s, introduced probabilistic models that could learn entity recognition patterns from annotated training data. Maximum Entropy Models, Support Vector Machines, and particularly Conditional Random Fields became the dominant approaches during this period. CRF based systems such as Stanford NER[4] demonstrated the effectiveness of sequence labeling formulations and established many of the evaluation standards and feature engineering practices that influenced subsequent research.

The neural revolution in NER began with the introduction of neural network architectures capable of learning distributed representations of words and contexts. Early neural approaches utilized feedforward networks with word embeddings, but the breakthrough came with the adoption of recurrent neural networks, particularly Long Short Term Memory (LSTM) networks, which could effectively model sequential dependencies in text. BiLSTM CRF architectures [5], combined the representational power of neural networks with the structured prediction capabilities of CRFs, achieving state of the art performance across multiple benchmark datasets.

The transformer era, initiated by the introduction of the Transformer architecture and subsequently accelerated by models such as BERT[3], fundamentally changed the landscape of NER research. Pretrained language models demonstrated unprecedented performance improvements across virtually all NER benchmarks, leveraging massive amounts of unlabeled text data to learn contextual representations that could be fine-tuned for specific entity recognition tasks. The success of transformer based approaches established new performance standards and shifted research focus from architecture design to effective utilization of pretrained models.

2.3 Current State of the Art: LLMs, UniNER, GLiNER

The contemporary landscape of Named Entity Recognition is dominated by large language models that have achieved remarkable performance across diverse entity types and domains. Models such as GPT4, Claude, Gemini, and specialized variants demonstrate exceptional capabilities in few shot and zero shot entity recognition scenarios, often matching or exceeding the performance of systems specifically trained for NER tasks.

Large language models approach NER through generative paradigms, typically employing structured prompting techniques that instruct the model to identify and classify entities within given text. These approaches leverage the extensive world

knowledge acquired during pretraining to recognize entity types and relationships that may not have been explicitly encountered during training. The flexibility of prompt based approaches enables rapid adaptation to new entity schemas without requiring model retraining, making LLMs particularly attractive for applications involving diverse or evolving entity requirements. Google Langxtract is one of the latest open source packages by google which aids users by using LLMs on multiple information extraction tasks based on the users problems. For examples users could easily install the package and use any provider and model, a custom prompt along with well defined output schemas, it also provides functions to help in extracting information from long documents by chunking them into smaller parts.[6]

UniversalNER[7] represents a significant advancement in this direction, proposing a unified framework for named entity recognition that can handle arbitrary entity types through targeted distillation from large language models. The approach demonstrates that carefully designed distillation processes can transfer the entity recognition capabilities of large models to more efficient architectures while maintaining competitive performance across multiple benchmarks. UniversalNER achieves this through a two stage process involving negative sampling for entity types and knowledge distillation that preserves the nuanced understanding of entity boundaries and classifications learned by the teacher model.

GliNER[8] represents a cutting-edge architecture in Named Entity Recognition (NER), offering distinct benefits over earlier models. Notably, it facilitates the creation of small models specialized in multi label extraction. Its performance is impressively similar to that of other models, such as UniNER and various Large Language Models (LLMs). GliNER introduces a novel architecture that incorporates spans and BERT within the model structure, a detail we will explore further in subsequent sections.

Although LLMs are powerful and versatile for a multitude of tasks, their high computational cost, substantial size, and inherent privacy concerns often render them impractical for large scale production systems serving numerous users. These drawbacks frequently lead to inefficient use of resources and time. Consequently, the industry trend is shifting toward smaller, specialized models, often distilled from larger LLMs. Such models, including GliNER and UniNER, provide several advantages: they are lightweight, significantly more cost effective, maintain data privacy, and can be trained or fine-tuned entirely in house [9].

2.4 Problems with Current State of the Art: Cost, Tokens, and Operational Limitations

Despite their impressive performance capabilities, large language models face fundamental limitations that restrict their practical deployment in production environments. The most significant challenge involves computational costs, which scale dramatically with the size and complexity of deployed models. State of the art LLMs require substantial GPU resources for inference, with costs that can quickly become prohibitive for organizations processing large volumes of text or requiring real time entity extraction capabilities.

Token based pricing models employed by most commercial LLM providers introduce additional complexity for production deployment. The cost per processed document varies significantly based on text length, entity density, and the complexity of required entity schemas. For organizations processing millions of documents annually, these costs can exceed traditional IT infrastructure budgets by orders of magnitude, making LLM based solutions economically unfeasible despite their technical superiority.

To illustrate the economic challenges facing organizations implementing LLM based NER systems, the following table presents a detailed comparison of token costs and operational limitations across major providers:

Provider	Model	Tier	TPM	TPD	RPM	RPD	input $/Million	Output $/Million
Google	Gemini-2.5-flash-lite	0	250,000.00		15	1,000.00	0	0
Google	Gemini-2.5-pro	0	125,000.00		2	50	0	0
Cerebras	qwen-3-235b-a22b-instruct-2507	0	60,000.00	1,000,000.00	30		0	0
Cerebras	Llama 3.1 8b	0	60,000.00	1,000,000.00	30		0	0
OpenAI	gpt-4.1 nano	0	40,000.00		3	200	0	0
groq	moonshot/kimi-k2-instruct	0	10,000.00	300,000.00	60	1,000.00	0	0
groq	Llama 3.1 8b	0	6,000.00	500,000.00	30	14,400.00	0	0
deepseek	Deepseek-chat	1	9,999,999.00				0.03	0.42
groq	Llama 3.1 8b	1	250,000.00		1,000.00	500,000.00	0.05	0.08
Google	Gemini-2.5-flash-lite	1	4,000,000.00		4,000.00		0.1	0.4
Cerebras	Llama 3.1 8b	1	1,000,000.00		1,000.00		0.1	0.1
OpenAI	gpt-4.1 nano	1	200,000.00		500	10,000.00	0.1	0.4
Cerebras	qwen-3-235b-a22b-instruct-2507	1	1,000,000.00		1,000.00		0.6	1.2
Anthropic	claude haiku 3.5	1	50,000.00		50		0.8	4
groq	moonshot/kimi-k2-instruct	1	250,000.00		1,000.00	500,000.00	1	3
Anthropic	claude haiku 4.5	1	50,000.00		50		1	5
Google	Gemini-2.5-pro	1	2,000,000.00		150	10,000.00	1.25	10
OpenAI	Gpt-5	1	500,000.00		500		1.25	10
Anthropic	claude sonnet 4.x	1	30,000.00		50		3	15

Table 2.4.1 : Comprehensive LLM Provider Cost Analysis, Trier =0 (Free), Tier =1 (Lowest paid tiers), TPM = Tokens Per Minute, RPM = Requests Per Minute, RPD = Requests Per Day, TPD = Tokens Per Day, [26]

Rate limiting and token quotas imposed by API providers create operational bottlenecks that can severely impact system throughput and reliability. Production systems must implement sophisticated retry mechanisms, queue management, and fallback strategies to handle API failures, quota exhaustion, and variable response times. These requirements add significant complexity to system architecture and can introduce unpredictable delays in processing pipelines.

The free tier limitations presented in the table demonstrate another significant constraint for development and testing scenarios. Most providers impose restrictive daily or monthly quotas that make it difficult to conduct comprehensive evaluations or proof of concept implementations. For example, Anthropic's Claude models limit free

users to just 25 requests per day, which would exhaust the quota within minutes of testing a moderate scale NER application.

The dependency on external API services raises additional concerns regarding data privacy, vendor lock in, and service availability. Organizations in regulated industries may be prohibited from transmitting sensitive documents to third party services, while the reliance on external providers creates single points of failure that can disrupt critical business processes. Furthermore, the lack of fine tuning capabilities for most commercial LLMs prevents organizations from optimizing performance for domain specific entity types or incorporating proprietary knowledge bases.

Latency considerations present another significant challenge for real time applications. LLM inference times can vary substantially based on server load, model complexity, and request queuing, making it difficult to provide consistent response times for interactive applications. The unpredictable nature of LLM performance characteristics complicates capacity planning and service level agreement guarantees essential for enterprise deployment.

Context window limitations, while improving across providers, still constrain the types of documents that can be processed in single requests. Although Google's Gemini 1.5 Pro offers a substantial 2 million token context window, most other models remain limited to 128K to 200K tokens, requiring document chunking strategies that can compromise entity recognition accuracy across document boundaries.

The emergence of specialized inference providers like Cerebras and Groq demonstrates market recognition of these limitations, offering significant speed improvements and competitive pricing through custom hardware architectures. However, these providers typically support only a limited set of open source models, constraining options for organizations requiring specific model capabilities or proprietary features.

These comprehensive cost and operational analysis clearly demonstrate why alternative approaches that can achieve comparable performance while avoiding the limitations of direct LLM deployment have become increasingly attractive for production NER applications. The economic and operational constraints identified in this analysis provide the foundation for understanding why knowledge distillation approaches, such as those enabled by GLiNER and LoRA fine tuning, represent compelling alternatives for organizations seeking to deploy high quality NER systems at scale.

2.5 GLiNER: Architecture, Working Principles, and Evaluation

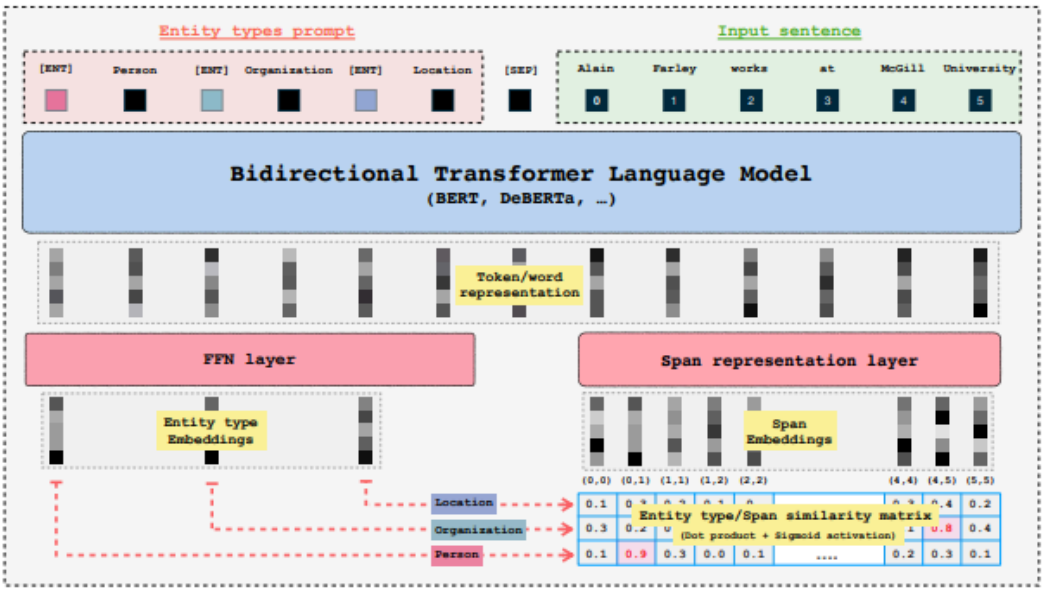

Fig 2.5.1: GLiNER architecture [8]

GLiNER (Generalist Lightweight Model for Named Entity Recognition) represents a paradigm shift in NER architecture design, offering a unified approach that can handle arbitrary entity types without requiring task specific model architectures or extensive retraining procedures [8]. The model's architecture fundamentally differs from traditional sequence labeling approaches by formulating NER as a span classification problem, enabling more flexible entity type specifications and improved handling of nested or overlapping entities.

The architectural foundation of GLiNER builds upon bidirectional transformer encoders, specifically leveraging encoder only architectures that can process entire documents in parallel while maintaining computational efficiency. The model employs a sophisticated span representation mechanism that generates contextual embeddings for all possible text spans within a given length threshold, enabling comprehensive entity candidate evaluation without sequential processing constraints.

The span representation process utilizes a combination of start and end token embeddings, enhanced with learned positional encodings that capture the relative positions and lengths of entity candidates. This approach enables the model to consider entity boundaries more explicitly than traditional BIO tagging schemes, leading to improved boundary detection accuracy and reduced label inconsistency issues common in sequence labeling formulations.

GLiNER's entity type conditioning mechanism represents another significant architectural innovation, allowing the model to dynamically adapt to arbitrary entity schemas through natural language descriptions of target entity types. Rather than learning fixed entity type embeddings, the model processes textual descriptions of entity types alongside input text, enabling zero shot recognition of entity types not encountered during training. This capability proves particularly valuable for applications involving evolving entity schemas or domain specific entity types that may not be well represented in general training corpora.

The model's training procedure incorporates negative entity sampling strategies that improve the discrimination between valid entities and non entity spans. By carefully selecting negative examples that resemble valid entities but lack appropriate semantic properties, the training process enhances the model's ability to identify entity boundaries accurately and reduces false positive rates in deployment scenarios.

Evaluation results across multiple benchmark datasets demonstrate GLiNER's competitive performance compared to specialized models while offering superior flexibility and computational efficiency. The model achieves particularly strong results in few shot scenarios where limited training data is available for specific entity types, highlighting the effectiveness of the span based formulation and entity type conditioning mechanisms.

The GLiNER family has expanded to include several specialized variants optimized for different use cases and performance requirements. GLiNER-Decoder introduces autoregressive generation capabilities that enable more sophisticated entity extraction patterns and improved handling of complex entity relationships [10], this was mostly inspired by its previous paper like the which has the idea of converting LLM to vector decoder, which uses the masked attention rather than the casual attention for decoders type models like GPT [11]. GLiClass extends the approach to general sequence classification tasks, demonstrating the broader applicability of the underlying architectural principles [12]. GLiDRE [13] and GLiREL [14] adapts the framework for document-level relation extraction and zero shot relation extraction, showcasing the potential for unified information extraction architectures .

GLiNER multitask further generalizes the approach to handle various information extraction tasks within a single model, reducing the computational overhead and deployment complexity associated with maintaining multiple specialized models [15]. GLiNER2 represents the latest evolution of the architecture, incorporating schema driven interfaces and improved efficiency optimizations that enable even faster inference and more flexible entity type specifications [16].

2.6 Common open source Datasets for NER benchmarks

Selecting suitable datasets for evaluating NER systems requires careful attention to several aspects, such as the variety of entity types, the quality and consistency of the annotations, the domain in which the data is written, and the extent to which the dataset represents challenges found in real world settings. Over time, several open source datasets have become standard benchmarks, partly because they offer a good balance of diversity and reliability.[3]

The CoNLL 2003 dataset is one of the most frequently used benchmarks. It is based on newswire text and includes four main entity types: persons, organizations, locations, and miscellaneous entities. It's clear annotation scheme and well organized structure make it a dependable reference point for comparing different NER systems. Because the dataset is large enough to support meaningful experimentation but still manageable, it remains popular for both academic research and practical model development.

OntoNotes 5.0 is another influential dataset, known for its broad coverage of multiple text genres, including news articles, conversational transcripts, and web content. It contains a wide range of entity categories, allowing researchers to examine how well NER models handle different writing styles and linguistic contexts. This diversity is particularly useful for studying generalization, which is an essential aspect of deploying NER systems in varied real-world environments.

More recent datasets, such as WNUT 17, focus on user-generated text from social media platforms. These datasets capture the noisy and highly informal nature of online communication, where spelling variations, abbreviations, and rapidly evolving terminology are common. They allow researchers to test how robust their models are when faced with unpredictable or unconventional language patterns.

In addition to these widely used benchmarks, this thesis makes extensive use of the MIT Movies dataset. Although it is less commonly used than CoNLL or OntoNotes, it offers several characteristics that make it particularly well suited for a detailed investigation of NER methods. The dataset contains thousands of annotated examples related to the movie domain and includes a wide range of entity types, such as titles, actors, directors, genres, years, characters, and plot-related concepts. This variety makes the dataset rich enough to study cross entity generalization while still remaining focused on a coherent domain.

The annotations in the MIT Movies dataset are consistent and clearly defined, which is especially important for evaluating model behavior under controlled conditions. The dataset is also large enough to support experiments involving different training sizes, data mixing strategies, and synthetic label generation, while still being computationally accessible. Another advantage is that the movie domain offers natural, everyday language that resembles many commercial applications but avoids the sensitivity or specialized terminology found in areas such as medicine or finance.

Because of these characteristics, the MIT Movies dataset serves as a strong foundation for the experiments in this thesis. It provides the right mix of diversity, clarity, and practicality needed to study the effects of active learning, synthetic labeling, and training data variations in a systematic and reproducible way.

Annotation schemes play a central role in NER because they define how entities are marked in text and how models learn to detect their boundaries. Traditional NER datasets rely on token level tagging formats such as BIO, IOBES, IO, IOE, or BIES. In these schemes, tokens at the beginning of an entity are typically tagged with a variation of “B”, tokens inside an entity receive “I”, and nonentity tokens are marked with “O”. More expressive schemes, such as IOBES or BIES, also distinguish between single token entities (“S”) and the final token in a multitoken entity (“E”). Different languages and domains sometimes benefit from different schemes, and earlier research has shown that certain formats can lead to higher performance depending on the characteristics of the dataset.

While these schemes differ in complexity, they all share the same purpose: to help the model identify exactly where an entity begins and ends. The choice of annotation scheme can affect error patterns, model generalization, and token boundary precision. Many of the common benchmark datasets discussed earlier, including CoNLL 2003 and OntoNotes, use BIO style token labeling.

In contrast to traditional sequence tagging methods, GLiNER does not use BIO, IOBES, or any token tagging scheme during training. Instead, it adopts a span based annotation approach. The model is trained using entity spans defined by their start and end positions together with the entity label. These spans refer to token indices during training, which allows the model to learn how segments of text correspond to specific entity types without relying on tag transitions such as B→I or I→O.[3]

A typical training annotation for GLiNER consists of a sentence and a list of entity spans. Each span only specifies the text of the entity, its label, and its character level or token level boundaries. For example, in the sentence **"Stade de France is located in Paris."**, the dataset may contain annotations such as:

"Stade de France", labeled as a **location**, with a start and end position marking the exact span of the entity.
"Paris", also labeled as a **location**, with its own start and end character offsets.

During training, these spans are converted internally into token level start and end indices, which makes the learning process more reliable and avoids many of the boundary related inconsistencies that occur in BIO style tagging.

When the model produces predictions, GLiNER returns character level spans rather than token tags. A typical prediction output includes the extracted entity text, its predicted label, the start and end character positions in the original sentence, and a confidence score. For example, given the sentence "Elton John performed in the United States in 2022.", a prediction might simply appear as:

"Elton John", labeled as a person, starting at character 0 and ending at character 10.
"United States", labeled as a location, starting at character 28 and ending at character 41.
"2022", labeled as a date, starting at character 45 and ending at character 49.

This span based output is straightforward, easy to interpret, and avoids the need to reconstruct entity boundaries from token tags. It also offers greater flexibility for multilingual inputs and irregular tokenization, since the final boundaries are always expressed directly at the character level.

By design, GLiNER's annotation approach provides a cleaner and more intuitive representation of entities compared to traditional tagging schemes. It avoids dependency on complex tag transitions, reduces sources of labeling error, and aligns closely with how entities are represented in modern span based NER models. This makes GLiNER particularly suitable for tasks that require precise boundary detection and supports more natural downstream processing.

2.7 Knowledge distillation with LoRA: Rationale, Comparison with Traditional Methods, and Default Configuration

The development of adaptation methods for large language models has gone through several stages, starting with full fine tuning and eventually moving toward more

efficient approaches such as transfer learning, knowledge distillation, and parameter efficient training. Each of these techniques was created to overcome limitations of the previous generation, especially as models grew larger and the computational cost of updating billions of parameters became difficult to manage.

Early NLP systems relied heavily on full fine tuning, where every parameter of a pretrained model was updated for each new task. This approach works well for smaller models and often leads to strong performance since the entire network can adapt to task specific patterns. However, as model sizes increased dramatically, full fine tuning became inefficient and impractical. Storing a separate copy of a large model for each task required huge amounts of memory, and training these models demanded significant computational resources. These limitations created a need for more efficient ways to reuse the knowledge already embedded in pretrained models.

Transfer learning addressed some of these challenges by allowing models to reuse general representations learned from large corpora and then adapt to downstream tasks with a smaller amount of training data. While transfer learning improved overall efficiency, it still depended heavily on full fine tuning, and therefore inherited many of the same storage and computation issues. Researchers began investigating whether it was necessary to update every parameter during task adaptation, or whether only a small portion of the model needed to change.

Around the same time, knowledge distillation emerged as a way to compress large models into smaller ones by training a lightweight "student" model to imitate the outputs of a larger "teacher" model. This technique reduced inference costs and made deployment easier, but it required training the student from scratch and often resulted in performance compromise, especially on complex tasks. Distillation also did not directly solve the problem of efficiently fine tuning large models for many tasks.

These challenges motivated the development of parameter efficient fine tuning (PEFT) methods such as adapter layers, prefix tuning, and prompt tuning. Instead of updating all model parameters, these techniques introduce small task specific modules while keeping most of the original model frozen. They reduced storage requirements and made it possible to maintain many task variants without duplicating the entire model. However, some methods struggled with stability or limited expressiveness, particularly on tasks requiring deeper adaptation.

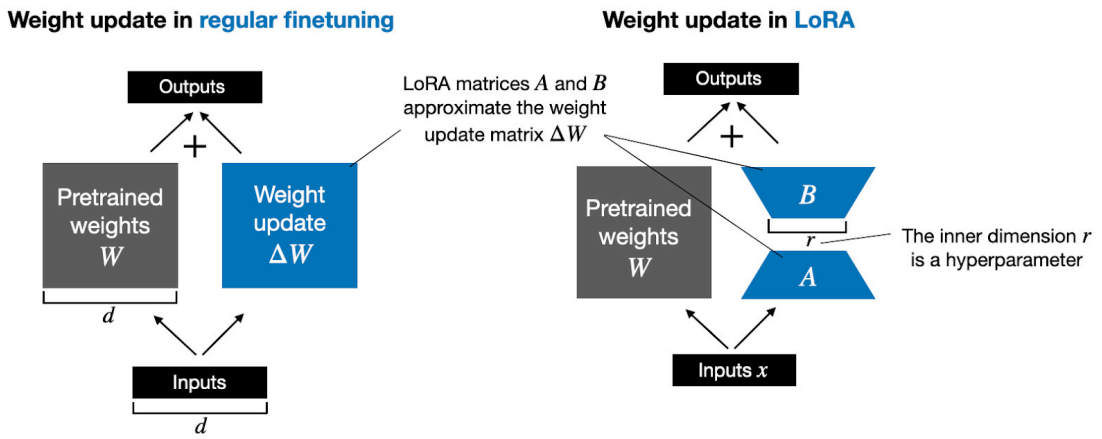

Fig 2.7.1: LoRA concept

Low Rank Adaptation (LoRA) represents a major step in the evolution of parameter efficient fine tuning. The key insight behind LoRA is that the changes needed to adapt a large model to a new task often lie in a low dimensional subspace. This means that instead of modifying full weight matrices which may contain millions of parameters task specific updates can be approximated using low rank matrices that are much smaller. LoRA introduces two trainable matrices, A and B, into the forward pass such that the weight update can be expressed as W + BA. Since B and A are low rank, they require only a small fraction of the parameters of the full model, yet they allow the network to learn the same task specific behavior.

This approach brings several practical advantages. Because LoRA keeps the original model weights frozen, only the new low rank matrices need to be stored for each task. This significantly reduces memory usage in scenarios where a system must support many adapted models at once. Training becomes faster and more stable because the number of trainable parameters is smaller, which also lowers the risk of overfitting on limited datasets. Another benefit is that LoRA adapters can be combined or swapped easily, offering flexibility that traditional fine tuning cannot achieve.

LoRA has been shown to match or surpass the performance of full fine tuning in many tasks, including NER, without the accompanying computational cost [17]. The method works particularly well for models where attention projections and feed-forward layers play a central role in capturing task specific relationships, as is the case in entity recognition. For this reason, LoRA has become a common choice for modern NER systems, including GLiNER [12], where adapter modules usually focus on attention and feed forward components. Typical configurations use ranks between 16 and 64, combined with scaling factors that help balance expressiveness and stability. The optimal choice of rank and target modules depends on several factors such as dataset size, complexity of entity types, and available computational resources.

In this thesis, LoRA provides an effective way to perform fine tuning while maintaining efficiency, stability, and reproducibility. Its combination of low memory usage, fast training, and strong performance makes it a practical solution for exploring NER tasks and comparing different training strategies.

2.8 Evaluation Metrics: With and Without Ground Truth, Confidence vs F1, Threshold Selection

Evaluating NER systems is a complex process that goes far beyond simply checking whether a model predicted the correct entities. A complete evaluation requires a clear annotation scheme, an appropriate evaluation strategy, and metrics that reflect both technical performance and real world reliability. Modern NER systems are typically assessed in two main ways: through traditional gold standard comparisons and through confidence based methods that support production monitoring when ground truth is unavailable.

The most common evaluation strategies for NER rely on comparing model predictions with the ground truth and are usually performed using either exact evaluation or relaxed evaluation. Exact evaluation requires the predicted entity boundaries and the entity type to match the gold standard perfectly. Even a slight mismatch in span or type is counted as an error. This strict method is used in well known benchmarks such as CoNLL 2003 and reflects scenarios where precision is critical. In contrast, relaxed evaluation gives the model partial credit when either the entity type or the span is correct, even if both do not match. This approach, used in earlier standards like MUC and ACE, captures cases where identifying the general region or category of an entity may still be useful for downstream tasks.

Traditional NER evaluation relies on precision, recall, and F1 score to measure how accurately the model identifies and labels entities. Precision reflects how many predicted entities are correct, recall measures how many of the real entities the model finds, and the F1 score balances both metrics into a single value. These metrics can be computed for individual entity types or combined across the dataset. Macro averaging treats all classes equally, while micro averaging weighs each prediction equally, regardless of class frequency. Both perspectives are useful, especially in datasets where entity distributions vary significantly. [3]

While these classical metrics remain essential, modern evaluation frameworks now recognize that they do not fully capture the complexity of real-world systems. For example, two models with identical F1 scores may behave very differently when deployed in production if their confidence estimates differ. As NER systems increasingly support semi-automated workflows, confidence-based evaluation has become an important component of system analysis. Confidence scores make it possible to route high certainty predictions directly into automated pipelines while flagging uncertain cases for manual review. This dynamic is particularly valuable in domains where the cost of false positives and false negatives differs.

The relationship between confidence and F1 performance is central to choosing an appropriate decision threshold. Different applications may require different trade offs. Some prioritise precision by keeping thresholds high, while others value higher recall and may accept more false positives. By examining performance across the full confidence spectrum rather than at a single threshold, it becomes possible to select operating points that align with practical requirements such as cost constraints, downstream error tolerance, and human review capacity.

In many real world scenarios, however, ground truth labels are not available during deployment. This limitation has led to alternative monitoring approaches such as consistency checks, shifts in the confidence distribution, and comparisons against historical model behaviour. These methods help detect performance degradation, data drift, or unexpected behaviour without requiring annotated data. Least confidence [18], Mean squared error and Maximum Normalized Log Probability[19] analysis of confidence distributions has also emerged as a promising way to identify low confidence or out of distribution inputs, which is especially useful in active learning settings. By identifying examples that systematically trigger low confidence, the system can prioritise them for annotation, improving model performance more efficiently than random sampling.

Recent research has also shown the value of examining performance beyond global metrics. For example, analysing entity level attributes such as span length, complexity, or density can reveal where models tend to struggle. Some models show strong performance on short entities but degrade significantly for longer phrases, while others may perform differently depending on language style or domain characteristics. Attribute based evaluation provides deeper insights that can guide model improvements and dataset design.

Finally, practical evaluation must extend beyond accuracy. Processing speed, memory usage, and scalability play a crucial role in determining whether a model can be deployed effectively. High performing models with slow inference times may not be suitable for real time applications, and systems that require excessive memory may be impractical for edge deployment. A complete evaluation therefore includes both accuracy metrics and operational metrics to ensure that performance gains do not come at the cost of usability.

Altogether, evaluation practices for NER systems have evolved to match the increasing sophistication of modern models. Classical measures like precision, recall, and F1 remain foundational, but contemporary evaluation now integrates confidence analysis, threshold optimisation, attribute based insights, cross domain testing, and practical deployment considerations. This broader perspective provides a more accurate picture of how a NER system will behave not only in research settings but also in real world, production level environments.

2.9 Manual Labeling Constraints: Role of Active learning

Manual annotation of training data is often one of the most significant bottlenecks in developing high quality NER systems. It demands both time and money: annotators must carefully read texts, identify entity boundaries, label correctly, and frequently consult detailed guidelines. These constraints become especially acute when many thousands of examples are required or when the entity schema and domain are complex.

Active learning offers a promising alternative to purely manual annotation workflows. In active learning, the system incrementally selects which unlabeled examples should be annotated next, typically focusing on the most informative or uncertain instances rather than labeling randomly. This targeted sampling can dramatically reduce the overall annotation budget while maintaining model performance. For instance, the work by Deep Active Learning for Named Entity Recognition [19] demonstrated that by combining deep learning with active learning, it was possible to approach state-of-the-art performance using only about 25 % of the original training data. Similarly, the empirical study by Deep Bayesian Active Learning for Natural Language Processing [18] showed that Bayesian active learning strategies using uncertainty estimates improved over random sampling and classic uncertainty heuristics across multiple tasks, including NER.

In an NER context, active learning typically operates by training an initial model on a small labeled seed set, then using an acquisition function (for example, least confident prediction, margin of confidence, or Bayesian disagreement) to pick additional

unlabeled instances for annotation. The process repeats iteratively until a labeling budget or performance target is reached. One example from Shen et al. introduces the "MNLP" measure (mean negative log probability)

In previous research, active learning for NER was almost always applied at the sentence level, where the model's uncertainty is calculated by looking at all tokens in the sentence. Methods such as MNLP, least confidence scoring, margin confidence, and Bayesian uncertainty have all been designed around this token-based view. However, GLiNER works differently. Instead of token level tagging, it predicts span level entities, meaning that uncertainty can also be measured at the level of each extracted entity rather than the whole sentence.

Because of this shift from tokens to spans, our study explores how traditional active learning strategies behave when applied to GLiNER's span based predictions. We experiment with several acquisition strategies, including MNLP, least confidence, random sampling, and confidence based measures such as mean squared error (MSE). The goal is to understand whether span level uncertainty leads to better sample selection and whether it can reduce labeling effort more effectively than the traditional sentence level approaches used in earlier work.

2.10 Synthetic Data Generation vs Synthetic Labels

The distinction between synthetic data generation and synthetic label generation represents a crucial conceptual framework for understanding different approaches to addressing annotation scarcity in NER applications. Synthetic data generation involves creating entirely artificial text documents that contain target entities, while synthetic label generation focuses on automatically annotating existing text with entity labels using automated systems such as large language models.

Synthetic data generation approaches, exemplified by systems such as ProgGen [20], GPT3Mix[21] and GuideX [22], create artificial documents designed to exhibit specific entity patterns and distributions. These methods offer the advantage of complete control over entity types, frequencies, and contextual patterns, enabling the creation of balanced training datasets that may be difficult to obtain through natural data collection. However, synthetic data generation faces significant challenges in replicating the linguistic complexity and contextual nuances found in real-world documents, potentially leading to models that perform well on synthetic data but struggle with authentic deployment scenarios.

ProgGen demonstrates a step by step approach to generating NER datasets using self-reflexive large language models that can iteratively improve the quality and diversity of generated examples. The system employs sophisticated prompting strategies that guide LLMs through structured generation processes, resulting in synthetic datasets that exhibit improved coverage of entity types and contextual patterns compared to simpler generation approaches.

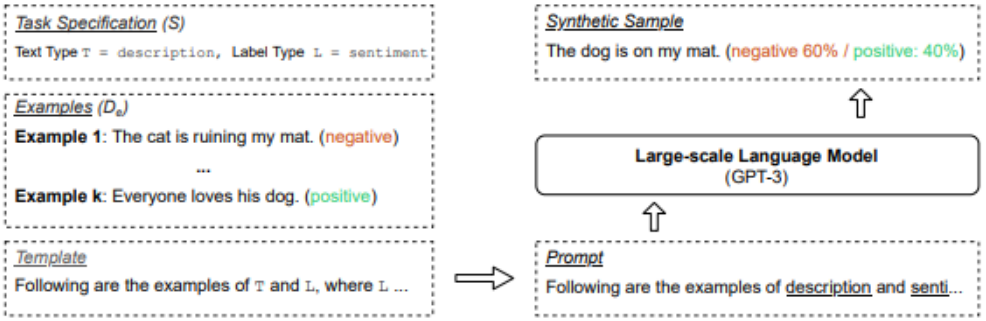

Fig 2.10.1 : GPT3 Mix [21]

GPT3mix employs a simple yet effective strategy: it creates novel data points by inputting two existing examples into a Large Language Model (LLM). The resulting generated points inherit characteristics from both source examples. This approach has been shown to significantly boost performance.

GuideX represents another significant advancement in synthetic data generation, focusing on zero shot information extraction scenarios where training data may be completely unavailable. The approach leverages guided synthesis techniques that can create diverse, realistic examples for arbitrary entity schemas without requiring extensive manual dataset development.

Synthetic label generation, by contrast, focuses on automatically annotating existing text documents using advanced NER systems, typically large language models that can accurately identify entities within authentic contexts. This approach preserves the linguistic authenticity and contextual complexity of real documents while reducing the annotation burden through automated labeling processes.

The advantages of synthetic label generation include the preservation of natural language patterns, contextual dependencies, and domain specific terminology that may be difficult to replicate in artificially generated text. Real documents contain the full complexity of human communication, including ambiguous references, implicit entity mentions, and complex syntactic structures that provide valuable training signal for robust NER systems.

However, synthetic label generation approaches must address quality control challenges, as automatically generated labels may contain errors or inconsistencies that could negatively impact model training. The evaluation of synthetic label quality requires sophisticated approaches that can identify systematic errors or biases in automated annotation systems while ensuring that the generated labels provide sufficient coverage of target entity types.[23]

The choice between synthetic data generation and synthetic label generation depends on multiple factors including the availability of unlabeled text in target domains, the complexity of required entity schemas, and the quality requirements for training data. Many successful approaches combine elements of both strategies, using synthetic data

generation to augment limited real data while employing synthetic label generation to scale annotation of available authentic documents.

Recent research has demonstrated that carefully curated synthetic labels can approach the quality of human annotation for many entity types, particularly when generated by state of the art large language models with appropriate prompting strategies. The effectiveness of synthetic labels appears to depend critically on the semantic complexity of target entities, with factual entities such as person names and locations being more amenable to automated annotation than subjective or context dependent entities such as sentiment expressions or implicit references.

The evaluation of synthetic labeling approaches requires comprehensive comparison with human annotation quality, analysis of error patterns and systematic biases, and assessment of downstream model performance when trained on synthetic versus human-generated labels. These evaluations must consider not only immediate accuracy metrics but also the robustness and generalization capabilities of models trained on different types of synthetic data.

The integration of synthetic and human annotation represents a promising direction that can leverage the strengths of both approaches while mitigating their individual limitations. Strategic mixing of synthetic and ground truth labels enables organizations to optimize annotation budgets while maintaining high quality training data, with the optimal[7]

Chapter 3. Methodology

3.1 Experimental Framework Overview

The experimental framework in this thesis is designed to study how a small span based model can approach or match large language models for named entity recognition under realistic cost and labeling constraints. The full implementation is available in the public repository at https://github.com/dcrey7/active_gliner , which contains all experiment scripts, configuration files and analysis notebooks.

The design follows insights from earlier work on active learning and annotation cost, which showed that uncertainty based sampling can reach strong performance with only a fraction of the labeled data compared to random selection[19]. Measures such as least confidence, mean negative log probability and related confidence based scores are used in this thesis to decide which examples are the most informative and should be labeled first, rather than treating all sentences as equally valuable.[18]

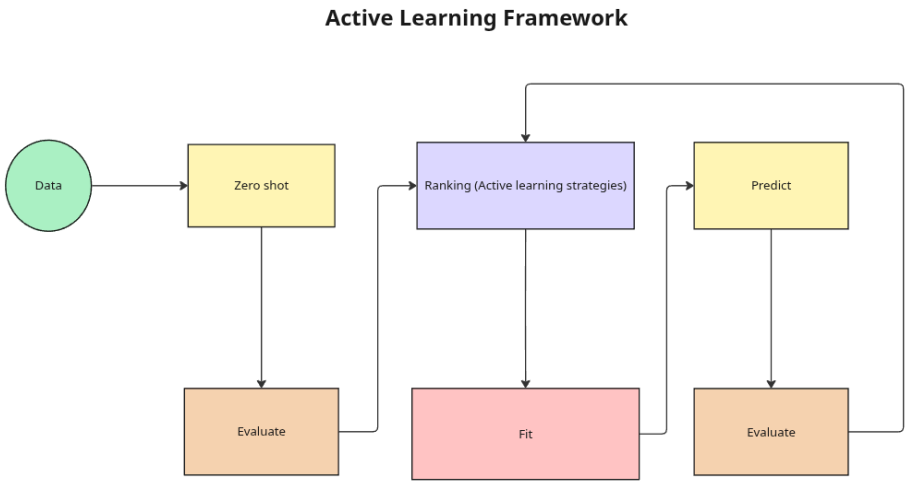

Fig 3.1.1: Active learning framework for GLiNER

The framework proceeds in four stages. First, zero shot baselines are established. The pretrained GLiNER model and the Gemma3 12B model are evaluated on the MIT Movies test set without any task specific training. This provides a clear reference for how a lightweight span based model and a large language model perform out of the box and defines the performance gap that later stages aim to close.

Second, confidence based ranking and active learning are introduced. GLiNER is run on the full training split to produce span level entity predictions with confidence scores. These span scores are aggregated into a single uncertainty value for each sentence using strategies inspired by the literature, including variants of least confidence, mean squared error and maximum normalized log probability, as well as a random baseline. The training sentences are sorted from most to least uncertain, which creates an ordering that simulates how an active learning system would choose which examples to label under a limited annotation budget.

Third, GLiNER is adapted using Low Rank Adaptation. LoRA updates are applied only to selected layers of the model, so that training affects a small fraction of the total parameters. A dedicated set of experiments is used to select a LoRA configuration and training schedule that achieves performance close to full fine tuning on the MIT Movies dataset. Using the ranked lists from the previous step, GLiNER is then fine tuned on growing subsets of the most uncertain examples, and its performance is measured on the held out test set. This stage shows how quickly a span based model improves as more labeled sentences are added and how parameter efficient tuning compares to full parameter updates.

Fourth, synthetic labels and mixed supervision are added. For the most uncertain sentences, Gemma 3 12B is run once as a label generator rather than as a production inference engine. Its outputs are validated, converted into the same training format as the human annotations, and then used to train GLiNER under three regimes. In the first regime, the model sees only ground truth labels. In the second, it sees only

synthetic labels. In the third, it sees mixtures of ground truth and synthetic labels at different ratios, which correspond to different choices of how much human effort is spent on annotation. Comparing these regimes shows how much human labeling can be replaced by large language model generated annotations without a large loss in accuracy.

All experiment scripts in the codebase are designed to be modular and configuration driven. The project structure follows common software engineering principles, including ideas from SOLID, so that each major concern such as data loading, model construction, active learning strategy, large language model backend and training loop is implemented as a separate component with a clear interface. Experiments are controlled mainly through small configuration dictionaries inside the run_experiments files, rather than a single monolithic program. This design makes it straightforward to change the model, switch between active learning strategies, adjust LoRA settings or modify mixing ratios, and to rerun the same experiment with different parameters. In practice, this modular structure significantly reduced the time needed to prototype, debug and repeat experiments.

All experiments were executed inside a Docker container on a dedicated machine with two NVIDIA RTX 3090 GPUs, more than 48 gigabytes of system memory and approximately one terabyte of storage, running Ubuntu. Access to this setup was made possible through the support of my mentor Philippe Fraisse, which allowed the full experimental suite to be run reliably and repeatedly.

Two families of large language models were used. Most experiments relied on a local setup using Ollama and Gemma 3 12B. This environment is beginner friendly, requires only a local download of the model and avoids external service limits. In this configuration Gemma 3 12B reaches around 70 percent F1 on the MIT Movies test set, which is a strong baseline for a relatively small model. Additional experiments used Qwen models served through the Cerebras platform. Qwen was chosen because it offers a generous free tier and simple access through an API key, but it still comes with rate limits and occasional failures. The codebase therefore includes explicit error handling, retry logic and quota handling to mirror real business scenarios where external LLM APIs can be slow, rate limited or temporarily unavailable. On the same dataset, the larger Qwen model achieves around 75% F1, which is higher than Gemma3 12B but comes at the cost of external dependency and less predictable latency. These two settings together provide a realistic comparison between local small models and hosted large models in an applied NER workflow.

3.2 Dataset Selection and Characteristics

This thesis uses the MIT Movies dataset as the single source of training and evaluation data. The dataset contains short, natural language queries about films, for example requests for movies by genre, year or actor. It is widely used in prior work on intent detection and slot filling and can be naturally treated as a named entity recognition task.

No.	entity	count_train	percentage_train	count_test	percentage_test
0	genre	4354	20.4	1117	20.9
1	actor	3220	15.1	812	15.2
2	year	2858	13.4	720	13.5
3	title	2376	11.2	562	10.5
4	rating	2007	9.4	500	9.4
5	plot	1927	9.0	491	9.2
6	average ratings	1869	8.8	451	8.4
7	director	1720	8.1	456	8.5
8	character	384	1.8	89	1.7
9	song	245	1.2	54	1.0
10	review	221	1.0	56	1.0
11	trailer	113	0.5	30	0.6

Table 3.2 : MIT movies Train and Test entity distribution

The version used in this work is split into train, development and test sets. The training split contains 9,774 examples and the test split contains 2,442 examples. On the training set there are 99,479 tokens and 21,294 entity mentions. On the test set there are 24,675 tokens and 5,338 entity mentions. This corresponds to an entity density of about 21 percent of tokens in both splits and an average of a little more than two entities per sentence. In practice this means that most user queries mention more than one piece of information, such as a genre and a year or a title and an actor, which makes the task more realistic and slightly more challenging than single slot datasets.

The dataset defines twelve entity types that cover both content and metadata. The most frequent entities are genre, actor and year, followed by title, rating, plot, average ratings and director. Less frequent but still important categories include character, song, review and trailer. In the training split, for example, genre accounts for about 20 percent of all entity mentions, actor for about 15 percent and year for about 13 percent. The remaining categories are more evenly spread, with director and rating each contributing around 8 to 9 percent and character, song, review and trailer together forming a small but non-trivial long tail. A very similar distribution is observed in the test set, which supports stable evaluation.

MIT Movies is a suitable choice for this thesis for three reasons. First, the domain is familiar and easy to interpret, which makes error analysis and qualitative inspection straightforward. Second, the entity schema is rich enough to cover both common and rare types, allowing us to study how active learning and synthetic labels behave for frequent categories like genre and actor as well as for low frequency labels like song or trailer. Third, the overall size is moderate. With just under ten thousand labeled training examples, it is large enough to support meaningful machine learning experiments but small enough that label cost and annotation efficiency remain realistic concerns. This balance makes the dataset a good testbed for studying how span based models, active learning strategies and mixed supervision can reduce labeling effort while preserving performance

3.3 Model Architecture and Selection

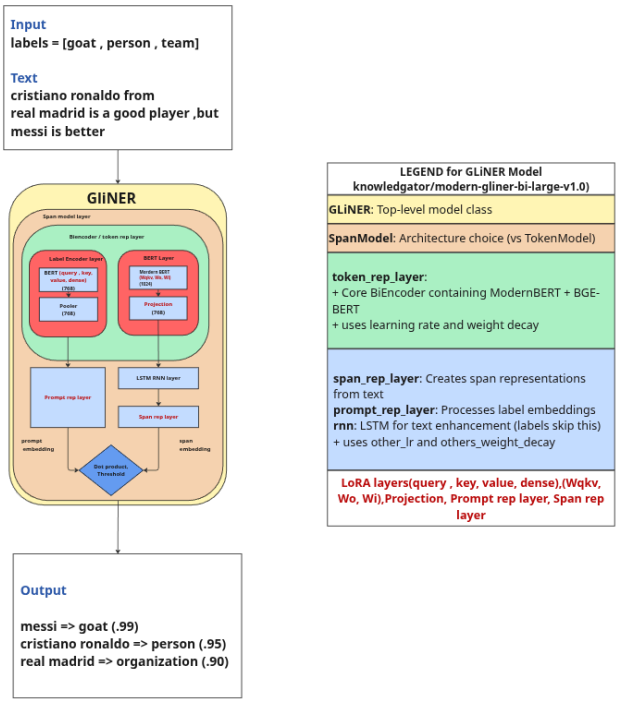

Fig 3.3.1: knowledgator/modern-gliner-bi-large-v1.0 internal architecture

The core model used in this thesis is the GLiNER architecture provided by Knowledgator, specifically the checkpoint published as knowledgator/modern-gliner-bi-large-v1.0 on the Hugging Face Hub. GLiNER is a span based model that scores candidate spans instead of tagging each token. Internally it uses a bi encoder style design, with one encoder processing the input sentence and another encoder representing the label descriptions. These representations are combined to score each possible span and label pair. The base model is built on a ModernBERT style backbone and supports long contexts, here configured up to 8,192 tokens, which is useful for information extraction tasks that may involve longer inputs.

This architecture was chosen because it is compact enough to run on commodity GPUs, supports flexible label prompts and aligns naturally with the active learning strategies used later in the thesis. Since GLiNER already produces span level predictions with confidence scores, it is straightforward to derive uncertainty measures for individual sentences and apply active learning on top of them.

On the MIT Movies dataset, three GLiNER configurations are considered. In the zero shot setting, the pretrained model is evaluated directly on the test set using only the list of entity types as a prompt. In this configuration it reaches an F1 score of about 46 percent, without any task specific training. When the model is fully fine tuned on all available training examples, updating all parameters, the F1 score increases to roughly

87 percent. A series of Low Rank Adaptation experiments then shows that it is possible to match and slightly exceed this performance while updating only a small fraction of the parameters. The best LoRA configuration, using the rank, alpha and training hyperparameters found in the hyperparameter search, achieves around 88 percent F1 on the same test set. This supports the claim that parameter efficient adaptation is sufficient for high quality NER on this task.

Metric	Gemma 3 12B	Qwen3-235B-A22B
Context Window	128k	256k
Artificial Analysis Intelligence Index	20	57
Terminal-Bench Hard (Agentic Coding)	1%	13%
τ^2-Bench Telecom (Tool Use)	11%	53%
AA-LCR (Long Context Reasoning)	7%	67%
Humanity's Last Exam (ARC AGI 1)	5%	15%
MMLU-Pro	60%	84%
GPQA Diamond	35%	79%
LiveCodeBench	14%	79%
SciCode	17%	42%
IFBench (Instruction Following)	37%	51%
AIME 2025	18%	91%
Blended Price USD / 1M tokens	$0.00	$2.63
Median Tokens/s	50	63
Latency – First Answer Chunk (s)	1.64 s	1.37 s
Total Response Time (s)	11.69 s	40.76 s
Reasoning Time (s)	0	31.51

Table 3.3.1: LLM stats from Artificial intelligence leaderboard [24]

Large language models are used mainly as teachers. The primary teacher model is Gemma 3 12B, an open source model released by Google and accessed locally through Ollama. Gemma 3 12B has a context window of 128,000 tokens and is reported in public benchmarks to be competitive with other open source models of similar size on general language and reasoning tasks. In the benchmark suite provided by the model host, Gemma 3 12B scores 20 on the Artificial Analysis Intelligence Index, 60 percent on MMLU Pro, 35 percent on GPQA Diamond and 18 percent on AIME 2025. On coding and tool use benchmarks it reaches 1 percent on Terminal Bench Hard, 14 percent on LiveCodeBench, 17 percent on SciCode, 11 percent on the telecom tool use benchmark and 37 percent on the instruction following benchmark. These scores place it clearly below very large frontier models, but still in a range that makes it useful as a strong teacher for narrow tasks such as NER.

From an operational point of view Gemma 3 12B is effectively free to run once downloaded. The blended price per million tokens is reported as 0 dollars, with a median speed of about 50 tokens per second, a first token latency of around 1.6 seconds and a total response time of roughly 12 seconds for the benchmark prompts. On the MIT Movies dataset, when Gemma 3 12B is used to generate synthetic labels

that are then compared against the ground truth, it reaches an F1 score of about 69 to 70 percent. This is well above the zero shot GLiNER baseline and demonstrates that a medium sized local model can provide high quality supervision without any direct token cost.

To test whether the conclusions depend on a particular large language model, an additional teacher is used. This is an open source Qwen3 model with approximately 235 billion parameters, accessed through the Cerebras inference platform under the name Qwen3 235B A22B. This model has a context window of 256,000 tokens and performs strongly on a wide range of benchmarks. In the same benchmark suite it scores 57 on the Artificial Analysis Intelligence Index, 84 percent on MMLU Pro, 79 percent on GPQA Diamond and 91 percent on AIME 2025, which indicates very strong reasoning abilities. On coding and tool use benchmarks it reaches 13 percent on Terminal Bench Hard, 79 percent on LiveCodeBench, 42 percent on SciCode, 53 percent on the telecom tool use benchmark and 51 percent on the instruction following benchmark. These numbers far exceed those of Gemma 3 12B and place Qwen among the strongest open source models currently available.

However, this additional capability comes with higher cost and more complex deployment. The blended price is reported as approximately 2.63 dollars per million tokens. The median generation speed is around 63 tokens per second, with a first token latency of about 1.4 seconds but a much longer total response time of roughly 41 seconds, driven by extended reasoning time. In the context of this thesis, Qwen is accessed through a remote API provided by Cerebras. This introduces rate limits, occasional timeouts and temporary errors, all of which are handled explicitly in the codebase through retry and quota management logic. This setup mimics real production environments where organizations rely on third party LLM APIs rather than local models.

When evaluated as a label generator on the MIT Movies dataset, the Qwen model achieves about 75 percent F1 against the ground truth, outperforming Gemma 3 12B by several points. At the same time, the difference between 70 percent and 75 percent F1 is much smaller than the large gaps seen on broad reasoning and coding benchmarks. This contrast is important. It shows that the huge advantage of very large models on general purpose benchmarks does not always translate into equally large gains for focused tasks such as movie domain NER, especially when a smaller model is already producing clean, structured labels.

Taken together, these three models define the main comparison axes in the thesis. GLiNER represents a compact span based model that is the main target for fine tuning and active learning. Gemma 3 12B serves as a practical open source teacher that can be run locally at zero marginal token cost. Qwen3 235B A22B, accessed through Cerebras, serves as a stronger but more expensive and operationally complex teacher. Comparing their behavior and costs provides a realistic view of the trade offs between small specialized models and large general purpose models in applied NER workflows.

3.4 Finetuning and LoRA configurations

Before running active learning or mixing ratio experiments, the training configuration for GLiNER and the LoRA setup were examined and tuned specifically for the MIT Movies dataset. GLiNER exposes a richer set of training parameters than typical models. Beyond the standard learning rate, weight decay and batch size, the training configuration uses separate parameter groups with distinct learning rates and weight decay values.

The main learning rate controls the ModernBERT encoder backbone, which includes all attention mechanisms and feedforward layers that process the input text. A separate others learning rate is applied to the task-specific components including span representation layers that mark entity boundaries, prompt representation layers that encode entity types, scoring layers that compute predictions and any fusion or decoder layers if present. This separation allows the pretrained encoder to be fine tuned conservatively while letting task-specific layers adapt more aggressively to the new domain.

Additional controls include gradient accumulation steps to simulate larger effective batch sizes, warmup ratio for the learning rate schedule and early stopping through a patience value measured in evaluation steps. The configuration also specifies num steps for the total training horizon, max grad norm for gradient clipping, focal loss parameters to handle class imbalance and logging and evaluation intervals for monitoring.

To select suitable values for these parameters, an automated hyperparameter search was conducted using Optuna, as implemented in the exp gliner best hyperparameters experiment. The search space covered both learning rates, both weight decay values, the warmup ratio, the batch size, the LoRA rank and the gradient accumulation factor. For each trial, GLiNER was trained for up to 2500 steps on the MIT Movies training data with evaluation on the full test set used both for early stopping and as the Optuna objective. Optuna's median pruning strategy was enabled so that trials whose performance fell clearly below the median of completed trials were stopped early, allowing more budget to be spent on promising regions of the search space. All runs were tracked with MLflow and the best performing configuration was saved to disk and then written back into the default training and LoRA configuration used throughout the rest of the thesis.

The resulting default training configuration for this work uses 2500 training steps, a batch size of eight with a gradient accumulation factor of one, a main learning rate around 2e-4 for the ModernBERT encoder, a smaller secondary learning rate for the task-specific layers and weight decay terms tuned separately for the different parameter groups. The warmup ratio is set to approximately 0.16, early stopping patience is fixed at seven evaluation windows and the maximum gradient norm is set to one. Focal loss is enabled with moderate alpha and gamma values to reduce the impact of easy examples. Together these choices define a stable training schedule that works well for the MIT Movies dataset and serves as the base configuration for all LoRA fine tuning runs.

LoRA itself is defined through two types of decisions. The first concerns the numerical hyperparameters such as the rank r, the LoRA scaling factor lora alpha and the dropout rate. These were included in the Optuna search space through the lora r

choices and the derived lora alpha values, while lora dropout was kept fixed at a moderate value. The second concerns where in the GLiNER architecture the low rank adapters are attached. GLiNER exposes many candidate modules including ModernBERT attention and feedforward layers, classic BERT style layers and task-specific components for span and prompt representations. Applying LoRA to all of them is not always beneficial and increases the number of trainable parameters.

To understand this tradeoff, a dedicated layer selection experiment was run in exp gliner best lora layers. The possible target modules were grouped into several interpretable sets. Task-specific layers included the span representation projections for start, end and output positions and the prompt representation layers. ModernBert layers comprised Wqkv, Wo, Wi and projection modules. BERT style layers covered query, key, value, intermediate.dense, output.dense and pooler.dense. An attention group combined Wqkv, Wo, query, key and value. Using these building blocks, eight configurations were tested:

- ModernBert plus Task
- BERT plus Task
- Attention plus Task
- All Layers
- ModernBert Only
- BERT Only
- Attention Only
- Task Layers Only

For each configuration a fresh GLiNER model was loaded, LoRA adapters were attached only to the modules listed for that configuration and the model was trained on the full MIT Movies training set using the tuned training schedule. After training the adapter was evaluated on the test set and the F1 score recorded. This loop produced a ranking of layer configurations and made it possible to identify which combinations of backbone and task-specific layers provide the best balance between flexibility and parameter efficiency. Because each run starts from a clean base checkpoint and uses a single explicit list of target modules, the experiment avoids accidentally nesting multiple LoRA adapters on the same layer within a single model instance.

The final LoRA configuration used in the main experiments combines the best performing rank and scaling values from the Optuna search with the best performing target module set from the layer selection study. In practice this means a moderate rank and alpha applied to a curated set of ModernBert, BERT and task-specific layers with dropout, gradient clipping and early stopping tuned to the MIT Movies domain. This configuration achieves high F1 scores while updating only a small fraction of the underlying GLiNER parameters and serves as the standard fine tuning setup for all subsequent active learning and synthetic labeling experiments.

3.5 Active Learning Selection Strategies

Active learning in this thesis follows the classic idea that a model should ask for labels on the examples it finds most informative rather than on a random subset of the

data. Earlier work, such as Deep Active Learning for Named Entity Recognition and Deep Bayesian Active Learning for Natural Language Processing, has shown that selecting examples by model uncertainty can reach strong performance with only a fraction of the labeled data compared to random sampling. Shen and colleagues introduced the MNLP measure, which turns token level probabilities into a single sentence level uncertainty score, and many studies also rely on least confidence scoring. These measures form the basis of the acquisition functions used here, and this thesis extends them with an additional metric. Most active learning work for NER assumes sequence taggers that make token level predictions. The model's uncertainty is then aggregated over all tokens in the sentence to decide which sentence to annotate next. GLiNER behaves differently. It predicts spans directly and assigns a confidence score to each candidate entity. To adapt active learning to this setting, the framework first computes span level scores for each predicted entity and then aggregates them into a single uncertainty value per sentence. The key design choice is how to turn a list of span scores into one number that reflects how unsure the model is about that example.

In the codebase, four uncertainty measures and one baseline are implemented in the selection.strategy module. For each sentence the following scores are computed from the list of span confidences.

Mean squared error, a metric introduced in this work as an alternative to classical MNLP and least confidence. It measures the average squared distance between each span of confidence and perfect confidence. Higher values indicate more uncertainty. Because the squared term penalises medium and low confidence spans more strongly, MSE captures not only how low the weakest span is, but also how many spans deviate from ideal confidence. This makes it particularly suitable for span based models where the number and quality of spans can vary from sentence to sentence.

- Minimum confidence, which takes the lowest span score in the sentence as a signal of weakness. Lower values indicate more uncertainty and mirror traditional least confidence sampling.
- Average confidence, which averages all span scores so that sentences with generally low confidence are treated as uncertain, even if no single span is extremely weak.
- Maximum normalized log probability, which is a span based version of the MNLP measure proposed by Shen. It averages the negative log scores and treats larger values as more uncertain.
- A random baseline, which assigns scores by random shuffling and ignores model confidence, is included for comparison.

The active learning experiments use a two step procedure. First, the base GLiNER model is applied to every example in the MIT Movies training split with a fixed confidence threshold. For each sentence the predicted entities and their confidence scores are stored. Sentences where GLiNER predicts no entities or where the ground truth has no entities are filtered out so that the ranking focuses on examples that are informative for entity learning. The uncertainty measures described above are then applied to each remaining sentence to produce one score per strategy. For each strategy the sentences are sorted from most uncertain to least uncertain, producing a ranked list.

Second, these ranked lists are used to simulate different annotation budgets. The exp_active_learning_confidence_strategies experiment script takes the top N examples for a range of subset sizes, converts their ground truth entities into the GLiNER training format and fine tunes the model using the tuned LoRA configuration described earlier. After training on each subset, the adapter is evaluated on the full test set and the resulting F1 score is recorded. Repeating this for all strategies and all subset sizes yields a family of learning curves that show how quickly each strategy improves performance as more labeled examples are added and how the new MSE based acquisition function compares to traditional MNLP and least confidence.

3.6 Synthetic label generation

Synthetic labeling in this thesis means using a large language model as an automatic annotator for the MIT Movies sentences and then training GLiNER on those labels. Instead of asking the LLM to answer user questions at runtime, we call it offline, collect its labels once and reuse them many times during fine tuning experiments. All of this is implemented in the llm and create_data modules of the active_gliner repository.

The pipeline starts from a set of sentences selected from the training split, usually those that GLiNER finds most uncertain according to the active learning strategies described later. For each sentence we build a prompt that lists the twelve entity types and clearly explains the required output format. The main template is the StandardPrompt. In plain language, it tells the model that it is an expert NER system, lists each entity type with a short description, shows the exact text to label and then presents a strict JSON format with two fields, text and entities. Each entity in the entities list is required to have an entity string and a list of types.

An example call looks like this in concept. The input text might be “show me comedy movies from 1990 with Tom Hanks”. The prompt asks the model to return a JSON object such as

```
{"text": "show me comedy movies from 1990 with Tom Hanks",
"entities": [
{"entity": "comedy", "types": ["genre"]},
{"entity": "1990", "types": ["year"]},
{"entity": "Tom Hanks", "types": ["actor"]}
]}
```

Fig 3.6.1 : LLM NER extraction example

For providers that support structured output, the codebase also offers a StructuredPrompt. This variant uses the same natural language instructions but attaches a JSON schema derived from a Pydantic model that describes the NERResponse structure. The backend can then enforce the schema directly, which

reduces the chance of malformed outputs. Both prompt styles share the same core idea; they simply differ in how strictly the output format is enforced by the LLM service.

Because the MIT Movies sentences are short, typically around ten tokens, most of the prompt length comes from the instructions and the list of entity types. In practice each request uses roughly one hundred input tokens on average. The LLM is encouraged to be explicit and to return all entities in a fully expanded JSON list. When several entities and their types are listed, plus repeated keys and punctuation, the responses easily reach several hundred tokens. Across all experiments, a typical call produces on the order of a few hundred to around seven hundred output tokens. These proportions are important for thinking about cost, because they show that even simple NER tasks can generate large outputs when strict JSON formats are used.

Every LLM response passes through a validation layer before it is accepted as a synthetic label. The validator first strips away any markdown formatting and extracts the JSON object. It then checks the structure against the NERResponse schema and verifies that each entity has non empty text, that the text actually appears in the original sentence, that all types belong to the allowed list and that no entity spans are obviously broken. Invalid responses are counted and stored for inspection, and if a response cannot be repaired the corresponding sentence is treated as having no synthetic entities. When a response is valid, the entities are kept as a list of {entity, types} pairs and passed to the conversion utilities. These functions locate each entity in the original sentence, map character offsets to token indices and produce the token level span representation that GLiNER expects for training.

The backends for Gemma and Qwen are instrumented to behave like realistic services. Each backend tracks the number of requests, input and output tokens, estimated cost in dollars, average latency and several error categories including connection errors, timeouts, rate limit errors and hard quota violations. When an error occurs the backend records it, retries with exponential backoff up to a configurable limit and, if needed, returns an empty label set for that sentence so that the experiment can continue. For the remote Qwen model, this error handling is especially important because actual API rate limits and daily quotas can be hit during large runs.

To avoid unnecessary calls, synthetic labels are cached on disk. Before a new labelling run begins, the loader functions search for existing files that already contain labels for the same strategy and subset size. If a matching file is found it is reused, and no new LLM requests are made. If only a smaller file exists, the backend is invoked only for the additional sentences needed. This caching strategy keeps the experiments reproducible and makes it possible to run multiple fine tuning studies on exactly the same synthetic labels without paying for them again.

Later sections of the methodology describe how these synthetic labels are combined with ground truth annotations and used to train GLiNER with LoRA. At this stage the key idea is that large language models are treated as reusable label generators. Prompts and schemas are designed to produce clean, structured outputs, validation and conversion steps ensure that only consistent entities reach the training pipeline and backend statistics track the practical costs and failure modes of relying on LLM services for annotation.

3.7 Training Data Composition and Mixing Ratios

This part of the methodology studies how different combinations of human ground truth labels and LLM generated labels affect GLiNER's performance. Throughout this thesis, ground truth always refers to human annotations, and synthetic labels always refer to labels proposed by an LLM. The goal is not to remove human annotators, but to understand how far we can extend a fixed human labeled core with additional synthetic labels before model quality starts to suffer.

All experiments in this section reuse the same ranked training pool produced by the active learning stage. For each acquisition strategy, the most uncertain sentences are collected up to a maximum of 2,500 examples. For every sentence in this pool there are two possible supervision sources. The first is the original human annotation from the MIT Movies dataset. The second is a synthetic annotation generated by Gemma 3 12B using the synthetic labeling pipeline described earlier and stored on disk after validation. In a real system this corresponds to a workflow where a certain number of sentences are labeled by humans and additional sentences are labeled by an LLM to increase coverage.

The first family of experiments, implemented in exp_confidence_gliner_llm_ft_f1, compares three training regimes that all rely on the same human labeled core. For each training size N, GLiNER is fine tuned on N sentences drawn from the ranked pool. In the ground truth regime the model is trained only on the human labels for these N sentences. In the LLM regime the human labels for these same sentences are temporarily replaced by the synthetic labels, which simulates the scenario where an organisation uses an LLM to annotate extra data, but always compares back to the human gold standard on the test set. In both cases the test set remains human labeled and is never replaced by synthetic labels.

The second family of experiments, implemented in exp_confidence_mixed_ft_f1, studies mixed supervision. For each requested training size N and each mixing ratio r in {0, 25, 50, 75, 100}, the experiment builds a training set from the top of the ranked pool. First it determines how many sentences are available with both human and synthetic labels, up to a maximum of 2,500. This number is the actual training size n_actual for that configuration. The first n_actual sentences are selected. Among these, a fraction r percent is assigned to use the human ground truth labels and the remaining 100 minus r percent is assigned to use the LLM labels. For example, a ratio of 25 means that one quarter of the training sentences use human annotations and three quarters use synthetic annotations. The underlying sentences are always the same; only the label source attached to each sentence changes.

For each configuration, the selected human and synthetic labels are converted to the GLiNER training format, combined into a single training set of size n_actual and used to fine tune GLiNER with the fixed LoRA configuration. After training, the adapter is evaluated on the fully human labeled test set and the F1 score is recorded together with N, n_actual, the mixing ratio, and the counts of human and synthetic examples. Repeating this process across all training sizes and mixing ratios produces a grid of experiments that shows how performance changes as more of the training supervision is delegated to the LLM while the evaluation standard remains purely human.

The experiment scripts save both detailed run level tables and compact pivot tables. The detailed tables record, for each run, the requested training size, the actual number of examples used, the mixing ratio, the respective counts of human and synthetic labels and the identifier of the trained adapter. The pivot tables group these runs by effective training size and mixing ratio and store the best F1 score observed for each combination, together with a small summary of how many sentences in that configuration were supervised by humans versus the LLM.

Methodologically, this design keeps the roles of humans and LLMs clearly separated. Human annotations define the gold standard and are always used for the test set. Synthetic labels are introduced only on the training side, as an additional source of supervision that can reduce the need to obtain more human labels beyond a chosen core set. By comparing models trained with different mixtures of human and synthetic labels at the same effective training size, the thesis quantifies how much extra human annotation can be saved while still maintaining strong performance on a fully human labeled evaluation set.

3.8 Evaluation Metrics and Framework

The evaluation framework in this thesis is designed to answer two related questions. First, how well do GLiNER and the LLMs match human annotations on the MIT Movies test set. Second, how confident are their predictions and where do they tend to make mistakes. To do this, the framework combines standard ground truth based evaluation with confidence based analysis that does not always require labels. All experiments, whether zero shot or fine tuned, are evaluated using the same tools in the evaluate_model module.

At the core is evaluate_with_ground_truth. This function compares predicted entities against human annotations for a given dataset, typically the MIT Movies test set or a selected subset of training sentences. Predictions are provided in character level format with start and end offsets, labels and optional confidence scores. The function converts these predictions to token level spans aligned with the tokenized text and then compares them to the gold spans. It computes entity level precision, recall and F1, both per entity type and in micro and macro form. It also tracks example level accuracy, counting how many sentences are labelled completely correctly versus partially or incorrectly. The output includes a detailed classification report with true positives, false positives and false negatives per entity type, as well as overall F1 expressed as both a fraction and a percentage.

When confidence scores are available, typically for GLiNER predictions, the same function also performs a confidence analysis. It aggregates scores for all predicted entities, computes an overall average confidence and builds confidence bins for each entity type, such as 0 to 25, 26 to 50, 51 to 75 and 76 to 100 percent. If true positive and false positive scores are supplied, it creates separate confidence distributions for correct and incorrect predictions. This helps to understand whether the model is well calibrated, for example whether high confidence predictions are usually correct and whether errors cluster at particular confidence levels.

The complementary function evaluate_without_ground_truth is used when only predictions are available and no labels exist, for example on unlabeled data or when exploring the behaviour of a newly fine tuned model. It checks that predictions contain confidence scores and then performs a purely confidence based analysis. It reports the overall average confidence, the total number of predictions and the same type of confidence bins as in the labeled case. It also identifies examples with unusually high or low average confidence, which can be useful for manual inspection or for defining thresholds in downstream systems.

Several experiment scripts build on this evaluation framework. The zero shot and fine tuned baseline experiments for GLiNER and the LLMs use evaluate_with_ground_truth to report F1 on the full MIT Movies test set. The threshold analysis experiments apply the same function at different confidence thresholds to study precision recall trade offs and to select suitable operating points. The confidence comparison experiments that focus on the hardest examples reuse evaluate_with_ground_truth on small subsets of low confidence sentences, allowing a direct comparison between GLiNER and LLM predictions when the task is most challenging. The active learning and mixing ratio experiments always finish by evaluating the trained adapters on the held out human labeled test set with the same metrics, which makes their results directly comparable.

By separating conversion, scoring and confidence analysis into clear steps, the evaluation framework ensures that all models and training regimes are judged under consistent conditions. Standard metrics such as precision, recall and F1 provide headline numbers for each experiment, while the deeper reports on entity level errors, example level accuracy and confidence distributions support detailed analysis of where and why different configurations succeed or fail.

Chapter 4. Results

4.1 Entity Type Performance Breakdown

The first set of results compares the zero shot GLiNER model with the LoRA fine tuned version on the MIT Movies test set. Both models are evaluated with the same framework described in the methodology, using human annotations as ground truth.

No	entity_type	0-25%	26-50%	51-75%	76-100%
0	genre	0	0	550	503
1	year	0	0	88	200
2	plot	0	0	239	329
3	average ratings	0	0	4	5
4	actor	0	0	158	484
5	title	0	0	19	11
6	song	0	0	13	25
7	character	0	0	143	174
8	rating	0	0	219	360

9	review	0	0	49	36
10	director	0	0	107	196
11	trailer	0	0	19	1
12	**total**	**0**	**0**	**1608**	**2324**

Table 4.1.1: Baseline Gliner model confidence analysis on MIT movies test set

No	entity_type	precision	recall	f1	tp	fp	fn	support	avg_confidence
0	genre	0.59	0.55	0.57	619	434	498	1117	0.79
1	year	0.71	0.28	0.41	205	83	515	720	0.88
2	plot	0.25	0.29	0.26	140	428	351	491	0.78
3	average ratings	0.11	0.00	0.00	1	8	450	451	0.54
4	actor	0.91	0.72	0.81	587	55	225	812	0.83
5	title	0.07	0.00	0.01	2	28	560	562	0.6
6	song	0.55	0.39	0.46	21	17	33	54	0.86
7	character	0.22	0.78	0.34	69	248	20	89	0.89
8	rating	0.37	0.43	0.40	217	362	283	500	0.77
9	review	0.07	0.11	0.09	6	79	50	56	0.81
10	director	0.95	0.63	0.76	289	14	167	456	0.8
11	trailer	0.20	0.13	0.16	4	16	26	30	0.58
12	**micro_avg**	**0.55**	**0.40**	**0.47**	**2160**	**1772**	**3178**	**5338**	**0.81**

Table 4.1.2: Baseline Gliner model classification report on MIT movies test set

In the zero shot setting, GLiNER reaches an overall F1 score of 46.6 percent. Example level accuracy is about 13 percent, meaning that only a small fraction of sentences have all entities predicted correctly. Entity level accuracy is around 40 percent. The model makes 3,932 entity predictions in total, with an average confidence of about 0.77. Most predictions fall into the higher confidence bins, with over half of all entities scored between 76 and 100 percent confidence, even though many of these are wrong. The classification report shows that performance varies strongly by entity type. Actor and genre already achieve reasonable F1 scores, but several labels such as average ratings, title, review and trailer have very low recall and F1. This pattern indicates that the pretrained model has some useful prior knowledge, especially for common entity types, but struggles with rarer or more domain specific categories.

No.	entity_type	0-25%	26-50%	51-75%	76-100%
0	genre	0	0	93	1077
1	year	0	0	81	739
2	plot	0	0	121	362
3	average ratings	0	0	116	386
4	actor	0	0	75	768

5	title	0	0	235	470
6	song	0	0	27	52
7	character	0	0	40	102
8	rating	0	0	52	486
9	review	0	0	145	47
10	director	0	0	35	399
11	trailer	0	0	10	35
12	**total**	**0**	**0**	**1030**	**4923**

Table 4.1.3: Finetuned Gliner model confidence analysis on MIT movies test set

No	entity_type	precision	recall	f1	tp	fp	fn	support	avg_confidence
0	genre	0.90	0.94	0.92	1052	118	65	1117	0.92
1	year	0.83	0.95	0.89	682	138	38	720	0.93
2	plot	0.73	0.72	0.73	355	128	136	491	0.84
3	average ratings	0.79	0.88	0.83	395	107	56	451	0.89
4	actor	0.89	0.92	0.91	750	93	62	812	0.89
5	title	0.69	0.87	0.77	487	218	75	562	0.89
6	song	0.53	0.78	0.63	42	37	12	54	0.89
7	character	0.47	0.75	0.58	67	75	22	89	0.91
8	rating	0.80	0.86	0.82	428	110	72	500	0.93
9	review	0.14	0.48	0.22	27	165	29	56	0.69
10	director	0.91	0.87	0.89	395	39	61	456	0.93
11	trailer	0.58	0.87	0.69	26	19	4	30	0.96
12	**micro_avg**	**0.79**	**0.88**	**0.85**	**4706**	**1247**	**632**	**5338**	**0.9**

Table 4.1.4: Finetuned Gliner model classification report on MIT movies test set

After fine tuning with the LoRA configuration optimised for MIT Movies, GLiNER's performance changes substantially. The overall F1 score increases to approximately 85.4 percent. Example level accuracy rises to about 57 percent, and entity level accuracy to around 88 percent. The fine tuned model makes 5,953 predictions with a higher average confidence of about 0.86. The confidence distribution shifts so that the large majority of entities fall into the 76 to 100 percent bin, and this time the high confidence predictions are usually correct, which suggests much better calibration.

At the level of individual entity types, the gains are clear. Genre, year and actor all move into the high nineties or high eighties for F1, with both precision and recall above 0.88. Difficult labels such as average ratings, plot, character and trailer, which had low recall and weak F1 in the zero shot case, now reach solid F1 values in the 0.7 to 0.8 range. Even low frequency categories like song and review improve, though review remains the most challenging class. The micro averaged precision and recall for all entities converge at about 0.84, showing that the fine tuned model is both accurate and balanced in terms of false positives and false negatives.

These baseline results establish three important facts for the rest of the thesis. First, the zero shot GLiNER model provides a meaningful but incomplete starting point, especially for common entity types. Second, LoRA based fine tuning on the MIT Movies training data is sufficient to close most of the gap to a strong supervised model, without updating all parameters. Third, the detailed per label metrics and confidence distributions provide a clear reference point against which the later active learning and mixed supervision experiments can be interpreted.

4.2 Confidence Analysis and Threshold Selection

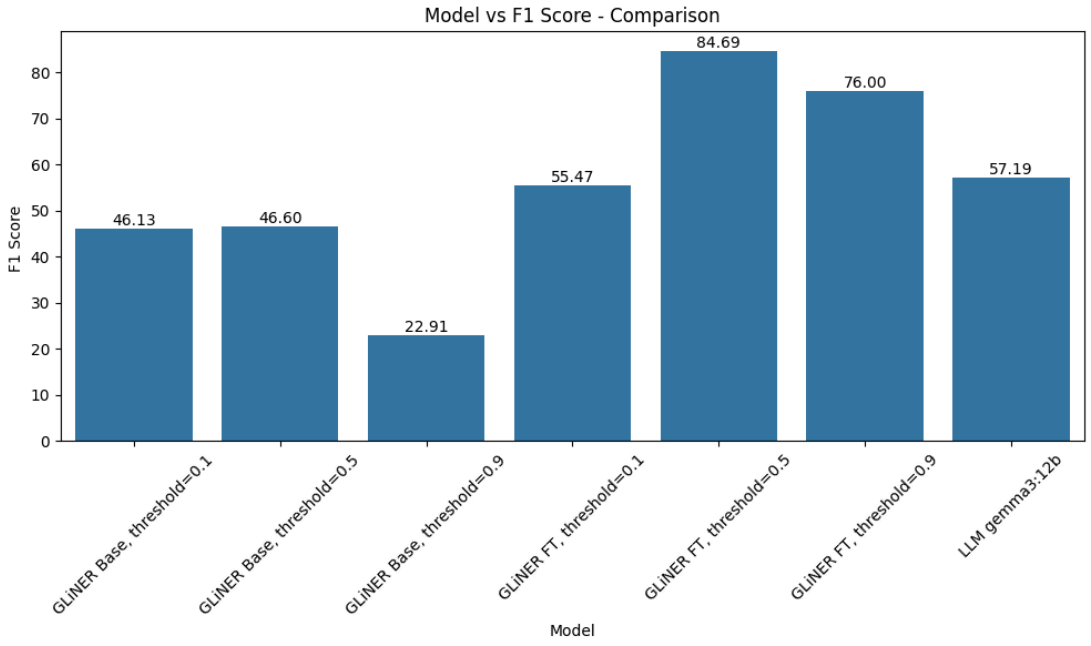

Fig 4.2.1 : F1 vs Models on MIT movies test set

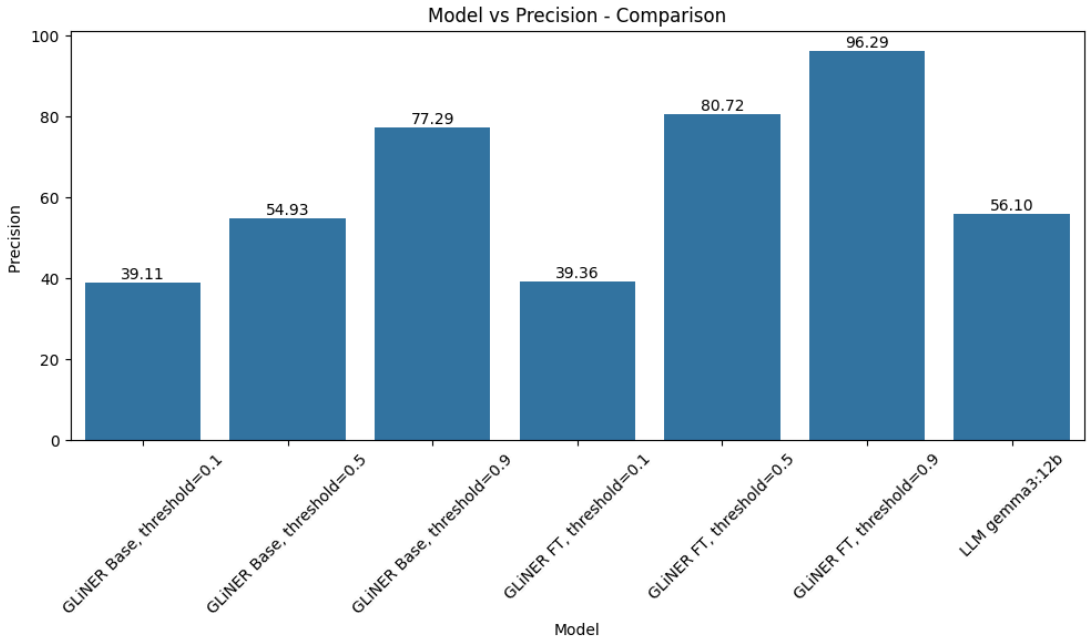

Fig 4.2.2 : Precision vs Models on MIT movies test set

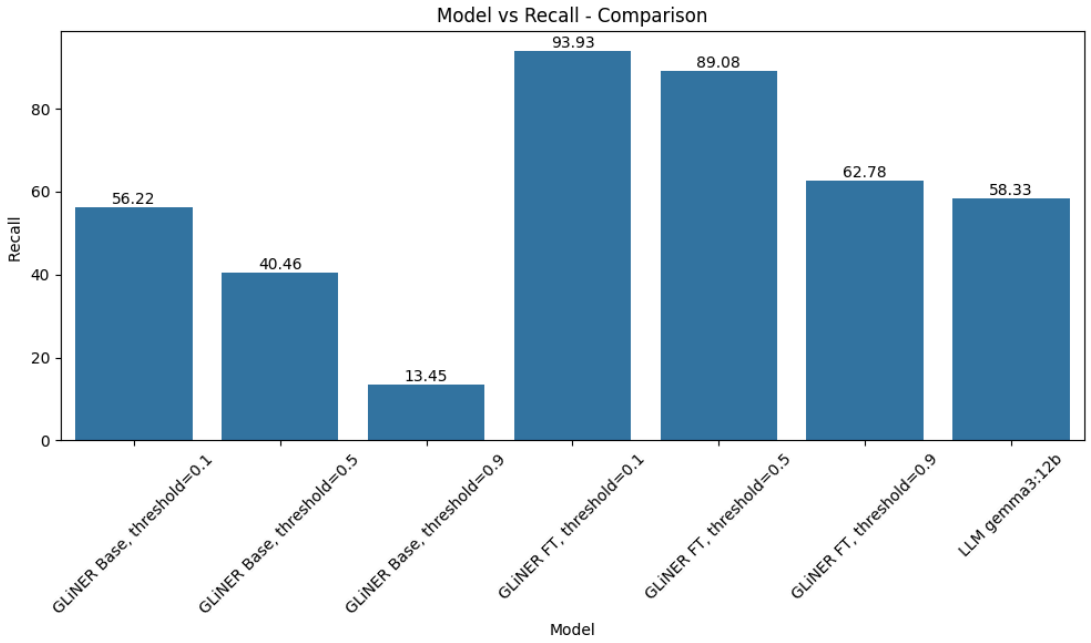

Fig 4.2.3 : Recall vs Models on MIT movies test set

Beyond overall F1, this thesis also examines how reliable GLiNER's confidence scores are and how different confidence thresholds change its behaviour. GLiNER assigns a confidence value between zero and one to every predicted entity. By setting a threshold on this value, the system can choose to accept only predictions above a certain confidence, which directly trades recall for precision. Lower thresholds keep more entities and favour recall, while higher thresholds keep fewer but more reliable entities and favour precision.

For the zero shot model, confidence analysis shows that many predictions are already assigned relatively high scores, even when they are incorrect. A large share of entities fall into the upper confidence bins, but the earlier classification report reveals that the overall F1 is still modest. This means the base model tends to be overconfident, especially on difficult entity types. As a result, raising the threshold does increase precision, but at the cost of discarding a significant number of correct entities and pushing recall down quickly.

After LoRA fine tuning, the picture changes. The average confidence increases and the distribution of scores shifts further towards the higher bins, but now this shift is backed by much better accuracy. High confidence predictions are correct far more often, and the separation between true positives and false positives in the confidence space becomes clearer. When we sweep the threshold, moderate values such as around 0.5 deliver a balanced operating point where both precision and recall are strong and the overall F1 is close to the maximum observed. Very low thresholds still inflate recall at the expense of precision, while very high thresholds produce extremely precise but overly conservative predictions that miss many entities.

Large language models such as Gemma 3 12B do not expose per entity confidence scores in the same way. Their outputs are converted into entity spans and evaluated once against the test set, but there is no natural threshold to tune. This means they

appear as a single point in the space of precision and recall, while GLiNER can be moved along a curve by adjusting its threshold. On the MIT Movies task, the fine tuned GLiNER model with a suitable threshold is able to reach a region where precision, recall and F1 are competitive with or better than the LLM baseline, while also providing the flexibility to shift towards higher precision or higher recall depending on application needs.

In summary, the confidence analysis shows that fine tuning does not only improve GLiNER's raw F1 score, it also makes its confidence estimates more meaningful. The zero shot model is useful as a starting point but overconfident, whereas the fine tuned model supports practical threshold selection. This ability to control the trade off between precision and recall through a simple threshold, while maintaining strong performance on a fully human labeled test set, is an important property for deploying GLiNER in real systems.

4.3 Best hyperparameters and LoRA configuration for training

A systematic hyperparameter search was carried out to find a configuration that is both accurate and parameter efficient. The search varied the main learning rate, the secondary learning rate for non LoRA parameters, batch size, weight decay, warmup ratio, gradient accumulation steps and the LoRA rank. The objective was to maximise test performance while keeping the number of trainable parameters as low as possible. The best setting reached an F1 score of around 88.5 while updating only about four percent of the model parameters, which makes it suitable for realistic deployments.

In condensed form, the final training setup can be expressed as a compact configuration dictionary. An illustrative example is

```
{
'num_steps': 2500,
'save_strategy': 'no',
'train_batch_size': 12,
'gradient_accumulation_steps': 1,
'learning_rate': 0.000214,
'others_lr': 0.000085,
'warmup_ratio': 0.198,
'eval_steps': 100,
'save_steps': 100,
'logging_steps': 10,
'max_grad_norm': 1.0,
'weight_decay': 0.000134,
'others_weight_decay': 0.0203,
'focal_loss_alpha': 0.75,
'focal_loss_gamma': 1.0,
'patience': 7 }
```

Fig 4.3.1 : Training configuration example for Gliner

This shows that training ran for 2 500 optimisation steps with a batch size of 12, used a slightly higher learning rate for the LoRA parameters than for the remaining weights, applied moderate weight decay and warmup, clipped gradients at a norm of 1.0 and relied on early stopping with a patience of seven evaluations.

The corresponding LoRA setup can also be summarised by a dictionary. The final adaptation configuration was of the form

```
{ 'r': 16, 'lora_alpha': 32, 'lora_dropout': 0.1, 'bias': 'none', 'task_type':
'token_classification', 'target_modules': [...] }
```

Fig 4.3.2 : LoRA configuration example for Gliner

Here the rank of 16 and the scaling factor of 32 define the size and strength of the low rank updates, and the dropout of 0.1 provides regularisation. This configuration achieved the strong performance mentioned above while keeping the adapter footprint very small.

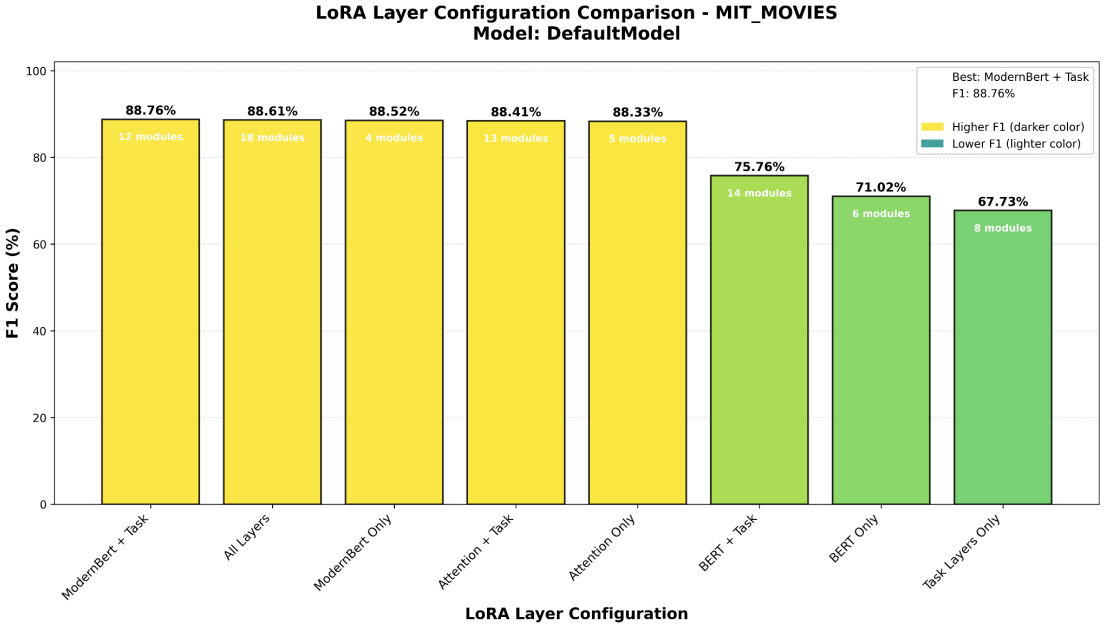

Fig 4.3.3: Best LoRA configuration

To determine which parts of the model should receive LoRA adapters, the layers were organised into simple target groups. In schematic form the groups can be written as

```
task_layers = [
'span_rep_layer.span_rep_layer.project_start.0',
'span_rep_layer.span_rep_layer.project_start.3',
'span_rep_layer.span_rep_layer.project_end.0',
```

```
'span_rep_layer.span_rep_layer.project_end.3',
'span_rep_layer.span_rep_layer.out_project.0',
'span_rep_layer.span_rep_layer.out_project.3',
'prompt_rep_layer.0', 'prompt_rep_layer.3' ]

modernbert_layers = [ 'Wqkv', 'Wo', 'Wi', 'projection' ]

bert_layers = [ 'query', 'key', 'value', 'intermediate.dense', 'output.dense',
'pooler.dense' ]

attention_layers = [ 'Wqkv', 'Wo', 'query', 'key', 'value' ]
```

Fig 4.3.4: LoRa target modules for knowlegator/ gliner-large-v1

Based on these lists, several LoRA layouts were defined through a configuration dictionary such as

```
lora_layer_configs = {
'ModernBert + Task': modernbert_layers + task_layers,
'BERT + Task': bert_layers + task_layers,
'Attention + Task': attention_layers + task_layers,
'All Layers': modernbert_layers + bert_layers + task_layers, 'ModernBert Only':
modernbert_layers,
'BERT Only': bert_layers,
'Attention Only': attention_layers,
'Task Layers Only': task_layers
}
```

Fig 4.3.5: LoRa layers grouped

Comparing these options showed that the all layers configuration, which attaches adapters to all four groups at once, consistently gave the best results. This layout therefore became the final choice for the target_modules field in the LoRA configuration

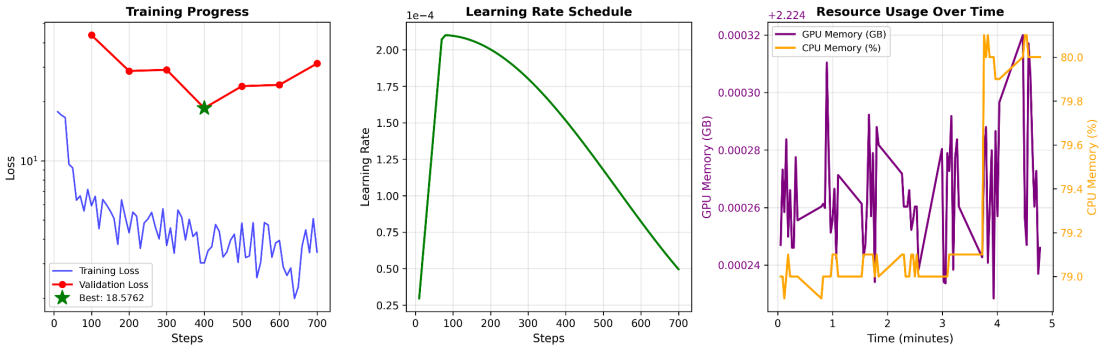

Fig 4.3.6: Training Dynamics

The training dynamics under this combined configuration were stable and easy to interpret. At the beginning of training the loss decreased quickly and the validation F1 rose sharply, indicating efficient use of the limited LoRA capacity. As training progressed the curves gradually flattened and approached a plateau, with only small fluctuations and no signs of divergence or severe overfitting. This pattern suggests that the chosen hyperparameters and layer layout form a well balanced regime for the considered task and justify using this configuration as the main reference in the subsequent experiments.

4.4 Active Learning Strategy Effectiveness

This part of the study evaluates how different active learning strategies affect data efficiency when fine tuning the span based model. Instead of selecting training sentences at random, the model first assigns a confidence based score to each example on the training set, derived from its span level predictions. The examples are then ranked according to this score, and the model is fine tuned with LoRA on progressively larger subsets of the highest ranked sentences. All other settings, including the LoRA configuration and training hyperparameters, are kept fixed so that the only difference between runs is the selection strategy.

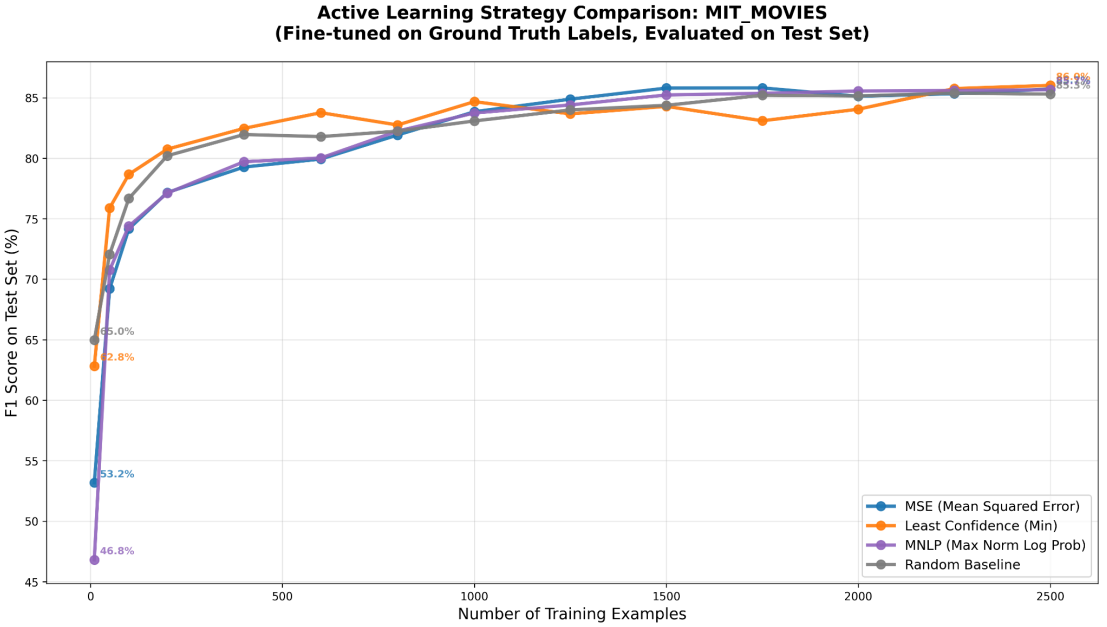

Fig 4.4.1 : Active Learning strategies comparison

Four strategies are compared. The least confidence strategy selects sentences where the model is generally uncertain, based on the lowest span confidence observed in the example. The MNLP strategy uses a normalised log probability score and prefers sentences whose predicted spans have low average certainty. The MSE strategy measures how far the span scores deviate from ideal confident predictions and again selects the most uncertain examples. Finally, a random strategy acts as a baseline that samples sentences without looking at model confidence. In all cases the selection is applied at the level of sentences, even though the underlying model operates on spans.

The experiment considers a range of annotation budgets, from as few as ten sentences up to 2500 sentences. For each budget, the model is fine tuned on the selected subset and evaluated on the full test set. The resulting learning curves show how F1 improves as more annotated examples are added. At very small budgets the curves are naturally noisy. For example, with ten labelled sentences the random baseline can occasionally match or even slightly outperform the informed strategies, simply because any tiny sample has a high chance of missing important entity types. However, once the budget reaches a few hundred sentences clear patterns emerge.

n_examples	min_f1	mnlp_f1	mse_f1	random_f1
10	62.82	46.80	53.17	64.97
50	75.89	70.77	69.21	72.06
100	78.67	74.39	74.16	76.67
200	**80.75**	77.12	77.17	**80.21**
400	82.47	79.72	79.27	81.96
600	83.77	**80.02**	79.93	81.79
800	82.75	82.25	**81.92**	82.24
1000	84.68	83.74	83.84	83.08
1250	83.67	84.41	84.88	84.00
1500	84.29	85.23	85.80	84.38
1750	83.10	85.37	85.82	85.20
2000	84.05	85.56	85.12	85.15
2250	85.76	85.60	85.34	85.38
2500	86.02	85.67	85.72	85.30

Table 4.4.1 : Active Learning strategies comparison, first to reach 80 f1

A first observation is that both the least confidence strategy and the random baseline reach an F1 score of about 80 percent at around 200 labelled sentences, while MNLP and MSE require more examples to cross the same threshold. Beyond this point the confidence based strategies gradually pull ahead of random selection. For subset sizes between roughly 400 and 1500 sentences, least confidence and MNLP in particular give higher F1 scores at the same budget than random sampling, indicating that the model benefits from focusing annotation effort on examples it finds difficult. When the budget approaches the upper end of 2500 sentences, all four strategies converge to a similar performance range above 85 percent, with the least confidence strategy ending slightly ahead of the others.

The shape of the learning curves is also informative. For all strategies the largest gains occur between about 50 and 400 labelled sentences, where each additional batch of annotated data produces a visible improvement in F1. Beyond roughly 1500 sentences the curves start to flatten and further gains become modest, which suggests that the most informative examples have already been incorporated. Overall, these results show that active learning based on model confidence can reduce the number of manually labelled sentences needed to reach a given performance level, and that a simple least confidence strategy is an effective and robust choice for span based named entity recognition in this setting.

4.5 Baseline and Finetune Performance Assessment

This section compares the span based model and the large language model on the most difficult training sentences and then studies how fine tuning changes this picture. Difficulty is defined through the model's own confidence scores. For each sentence in the training set the span model assigns a confidence value to its predictions. The sentences with the lowest confidence are treated as worst confidence examples. All evaluations in this section use the standard decision threshold of 0.5 for the span model. Throughout this section the large language model is a Gemma 3 12B model, which is used both as a direct NER baseline and as the source of synthetic labels for the fine tuning experiments

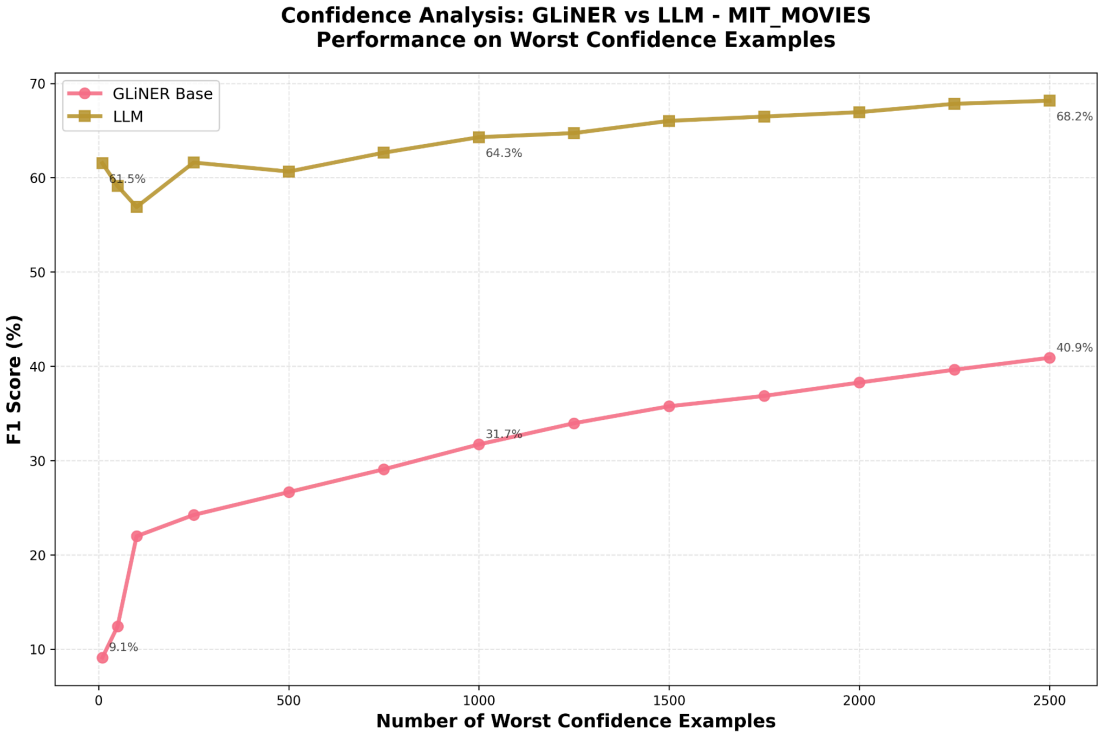

Fig 4.5.1 : Zero shot comparison between Gemma3 12b and Gliner Model on MIT Train set examples based on MSE strategy

The baseline experiment first measures how well the un fine tuned span model and the large language model perform on these worst confidence sentences. For each subset size, from 10 up to 2500 of the hardest sentences, the models are evaluated only on this subset using the available ground truth labels. The results show a clear gap. On the smallest subsets the span model reaches F1 scores in the range of about 10 to 25 percent, while the large language model already achieves around 60 percent. As the subset size grows the span model improves steadily and reaches roughly 40 percent F1 at 2 500 difficult sentences, but the large language model remains clearly ahead and stabilises around 68 percent. This confirms that, without fine tuning, the span model struggles especially on the cases where it is least confident, whereas the large language model is more robust even on these hard examples.

The second experiment keeps the same ordering of worst confidence sentences but now uses them for supervised fine tuning of the span model. Two variants are considered. In the first variant the model is fine tuned only on large language model generated labels for these difficult sentences. In the second variant it is fine tuned on the corresponding ground truth labels. In both cases the same LoRA configuration and training hyperparameters from the previous sections are used, and performance is measured on the full test set rather than on the difficult training sentences.

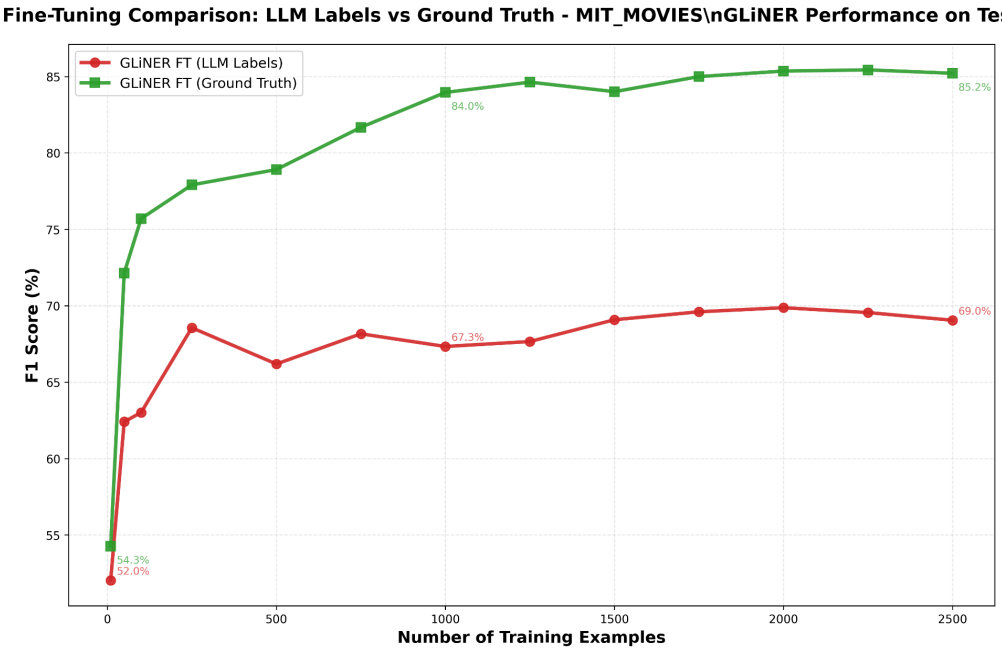

Fig 4.5.2 : Gliner Model Finetuned comparison between Ground truth and Gemma3 12b labels on MIT Train set examples based on MSE strategy and evaluated on Test set

The fine tuning results highlight a clear difference between the two label sources. When the model is trained only on large language model labels its test F1 starts around the low fifties for ten worst confidence examples and increases to roughly 69 percent when 2500 such examples are used. The curve is relatively flat and never exceeds the level of the large language model itself. In contrast, when the model is fine tuned on ground truth labels for the same difficult sentences, the test F1 already exceeds 70 percent with only 50 examples and climbs into the high seventies and low eighties as the number of training sentences reaches a few hundred. With around 2000 to 2500 ground truth labelled worst confidence examples the span model reaches test F1 scores in the mid eighties and clearly outperforms the large language model baseline.

Overall, these experiments show that the large language model is much stronger than the un fine tuned span model on the hardest sentences, but that a parameter efficient fine tuned span model can close and even invert this gap when trained on a relatively small number of carefully chosen ground truth examples. At the same time, training exclusively on large language model generated labels for these difficult cases does not match the performance of ground truth fine tuning and tends to saturate below the level of a well tuned span based system.

4.6 Mixed Ratio Training Results

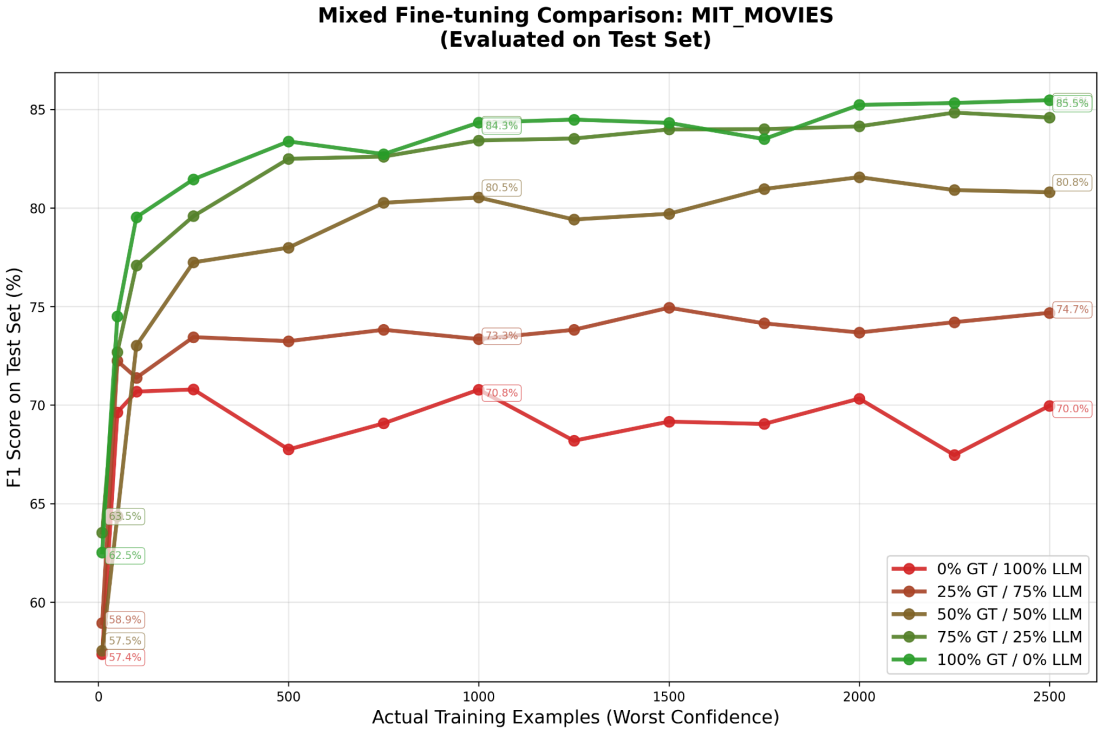

Fig 4.6.1 : Gliner Model Finetuned comparison between Ground truth and Gemma3 12b labels with different mixing rations on MIT Train set examples based on Min strategy and evaluated on Test set

This section studies how different mixtures of synthetic and ground truth labels affect the performance of the span based model. Throughout this experiment the large language model used to generate synthetic labels is Gemma3 12B. For each budget of worst confidence training sentences, the model is fine tuned with LoRA on a fixed number of examples, but the proportion of ground truth and Gemma generated labels is varied. The considered mixing ratios are 0 percent, 25 percent, 50 percent, 75 percent and 100 percent ground truth, with the remaining examples labelled by the large language model. Performance is always measured on the held out test set, not on the training sentences.

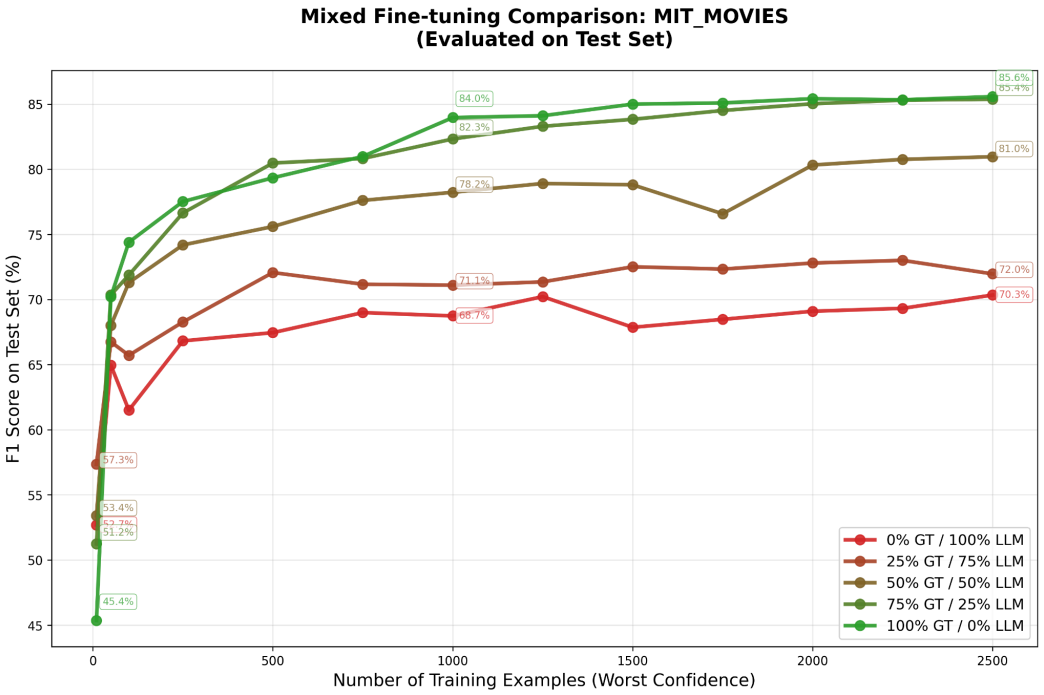

Fig 4.6.2 : Gliner Model Finetuned comparison between Ground truth and Gemma3 12b labels with different mixing rations on MIT Train set examples based on MSE strategy and evaluated on Test set

The results show a clear and intuitive pattern. When the model is trained only on synthetic labels, that is with 0 percent ground truth, the test F1 quickly rises into the high sixties as the number of worst confidence examples increases, but then saturates around 69 to 70 percent. Adding even a small proportion of ground truth labels improves performance. With 25 percent ground truth and 75 percent synthetic labels, F1 scores move into the low seventies for larger budgets, which is a noticeable gain but still clearly below the level of a fully supervised span based model.

The most interesting range is in the middle. A balanced mix of 50 percent ground truth and 50 percent synthetic labels already produces strong performance. For example, with a few hundred to around one thousand worst confidence examples, this setting yields F1 values in the high seventies and low eighties on the test set. Increasing the share of ground truth labels to 75 percent brings the model even closer to the best fully supervised baseline. Across most budget sizes, the 75 percent ground truth curve lies just below or on par with the 100 percent ground truth curve and often within a fraction of a percentage point of it once the number of examples exceeds roughly one thousand. This indicates that, in practice, a relatively modest fraction of human annotations can be sufficient if the remaining labels are provided by a strong large language model.

Purely supervised training with 100 percent ground truth labels remains the best choice when the highest possible accuracy is required and annotation resources are available. In that setting the model reaches F1 scores in the mid eighties for larger budgets of difficult examples. However, the gap between 75 percent and 100 percent ground truth becomes very small as the budget grows, while the difference between 0 percent and 50 percent ground truth remains substantial. This suggests that the choice of mixing ratio can be adapted to the needs of the application. Scenarios with strict quality requirements can favour high ground truth shares, whereas cost sensitive

settings can reduce human annotation effort by relying on balanced mixtures such as 50 to 75 percent ground truth combined with synthetic labels from Gemma 3 12B.

The results described above are based on the minimum confidence ranking of worst examples. An analogous experiment using an MSE based confidence score showed the same qualitative behaviour of the mixing ratios, which indicates that the conclusions are robust to the specific confidence metric used for selecting the difficult training sentences.

4.7 Cost Benefit Analysis

The economic viability of named entity recognition (NER) systems depends on balancing model accuracy with operational cost. This section evaluates the financial implications of using commercial large language models (LLMs) versus fine-tuned GLiNER models for large-scale NER. All cost estimates are based on current public API pricing from major providers and GPU compute costs in cloud environments. Commercial LLM pricing is structured per token, with input tokens encoding the task definition and the text to be analyzed, and output tokens representing extracted structured entities. For realistic NER workloads, a single extraction request typically requires approximately 1,000 input tokens and produces approximately 1,000 output tokens. Processing one million tasks therefore entails roughly one billion input tokens and one billion output tokens.

The table below summarizes the costs of processing one million NER tasks across major commercial models. Input and output token prices are kept exact, while cost per million tasks is shown as an approximation (≈) based on one million tasks each requiring 1,000 input and 1,000 output tokens.

Provider	Model	Input $/M	Output $/M	Approx. Cost per 1M Tasks	F1 Performance
OpenAI	GPT-4o	2.5	10	≈ €11,364	~98%
OpenAI	GPT-4o mini	0.15	0.6	≈ €682	~95%
Anthropic	Claude Haiku 4.5	1	5	≈ €5,455	~94–96%
Cerebras	Qwen 3 235B	0.6	1.2	≈ €1,636	~94–96%
Google	Gemini 2.5 Flash	0.1	0.4	≈ €455	~93–95%
DeepSeek	DeepSeek Chat	0.03	0.42	≈ €368	~93–94%

Table 4.7.1 : Costs per of different LLMs models on 1M tasks and f1 performance

These numbers follow directly from published pricing [25] [26] or Table 2.1. For example, GPT-4o mini costs ≈€682 per million tasks, computed as 1,000 million input tokens at $0.15/M plus 1,000 million output tokens at $0.60/M. Costs recur linearly with task volume, forming a continuous operational expense.

GLiNER reverses the LLM cost profile by concentrating expenses in a one-time training phase, after which inference is performed locally with no per-task API fees.

Many organizations already accumulate labeled data through routine business processes, reducing annotation costs. When approximately 1,000 existing labels are available, training cost is dominated by cloud GPU rental.

To ensure clarity and conservative estimates, this analysis rounds GPU training to a full hour, even though LoRA fine-tuning of BERT-scale encoders typically completes in about 30 minutes . An EC2 GPU instance (e.g., g5.xlarge) costs ≈€1 per hour in US regions[27], while a managed SageMaker GPU instance (e.g., ml.g5.xlarge) costs ≈€5 per hour. Synthetic examples may be added to augment the dataset; generating 1,000 synthetic labels with GPT-4o mini requires approximately one million input tokens and one million output tokens, costing ≈$1 using the pricing in Table above.

Configuration	Synthetic Labels	EC2 GPU Training (1 hr)	SageMaker GPU (1 hr)	Approx. Total (EC2)	Approx. Total (SageMaker)	F1 Performance
100% Ground Truth (2,000 labels)	€0	≈€1	≈€5	€0+€1≈€1	€0+€5≈€5	85–86%
50% GT + 50% Synthetic (1,000 + 1,000)	≈€1	≈€1	≈€5	€1+€1≈€2	€1+€5≈€6	87%

Table 4.7.1 : Costs of various training Gliner on Cloud

Let us assume that we need to Extract 1 million tasks per year using GPT 4o mini which amounts to €682 euros per year and then we assume that GLiNER Model performance degrades gradually due to domain drift, requiring periodic retraining to maintain accuracy. Prior research recommends monitoring data drift and retraining every six to twelve months depending on domain characteristics [28], [29] from which we get €2+€2= €4 year, Assuming two retraining cycles per year, GLiNER's annual operating cost remains negligible, since GliNER would have zero inference cost.

Year	GPT-4o mini	GLiNER (2 retraining cycles) EC2	Annual Savings
Year 1	€682	≈€4	≈€678 (≈99.9%)
Year 2	€682	≈€4	≈€678 (≈99.9%)
Year 3	€682	≈€4	≈€678 (≈99.9%)
3 Year Total	**€2,046**	**≈€12**	**≈€2,034 (≈99.9%)**

Table 4.7.3 : NER with LLM vs Gliner

These results underscore the substantial long term difference between continuous LLM based inference and GLiNER's fixed, usage independent training cost. Even with conservative rounding, GLiNER's three year cost is below ten euros, while an LLM based system processing one million tasks per year exceeds two thousand euros.

Chapter 5. Other Experiments and Trials

5.1 Alternative LLM Backends

In addition to Gemma, 3 other large language models were evaluated as potential backends for synthetic labelling and direct named entity recognition on the MIT Movies dataset. The first amd second was Qwen3 235B A22B 2507 instruct following (IF) and Qwen3 235B A22B 2507 reasoning(R), accessed through the Cerebras API The third was a 7 billion parameter Mistral model, deployed locally using the Mistral inference framework. This setup allowed a comparison between a very large cloud hosted model and a more lightweight model that can be operated on local hardware.

Both models produced qualitatively similar behaviour in terms of the types of entities they detected and the kinds of errors they made. However, their quantitative performance on the test set differed. Qwen3(IF) achieved an F1 score of around 75 percent and Qwen3 had around 77 F1 percent , which places it between the untuned span based model and the best fine tuned configurations described earlier. The locally deployed Mistral 7B model reached an F1 score of about 64 percent on the same evaluation, which is clearly weaker than Qwen3 but still competitive compared to traditional baselines.

These results suggest that the choice of large language model backend can be adapted to the constraints of the application. When cloud access and higher computational cost are acceptable, a very large model such as Qwen3 provides stronger synthetic labels and a better direct NER baseline. When deployment must remain local and resource efficient, a smaller model such as Mistral 7B offers a viable but less accurate alternative whose outputs can still be used in the same mixing and fine tuning framework.

5.2 Synthetic Data Generation Attempts and Prompting Strategy Variations

It is useful to distinguish clearly between synthetic labels and synthetic data. Synthetic labels refer to annotations produced by a language model for sentences that already exist in the original dataset. The text comes from human sources, but the entity labels are generated automatically. Synthetic data, in contrast, refers to completely new examples where both the text and the labels are created by the language model. The experiments in this section focus on the second case, that is, on fully synthetic text and labels.

To generate such data, the Gemma 3 12B model was prompted with a few shot set of MIT Movies examples, each consisting of a user query and its entity annotations. The model was asked to produce new movie related queries together with entity labels in the same format. To increase variety and avoid many near duplicates, the prompts included simple control variables. Following ideas from the GLiNER work and related approaches such as Guidex and ProGen, these variables specified attributes such as genre and country. Genre values included for example comedy, action and thriller, while country values included Japan, France, India and China. For each combination of these attributes the model was asked to create several new annotated sentences, leading to a reasonably diverse pool of synthetic data.

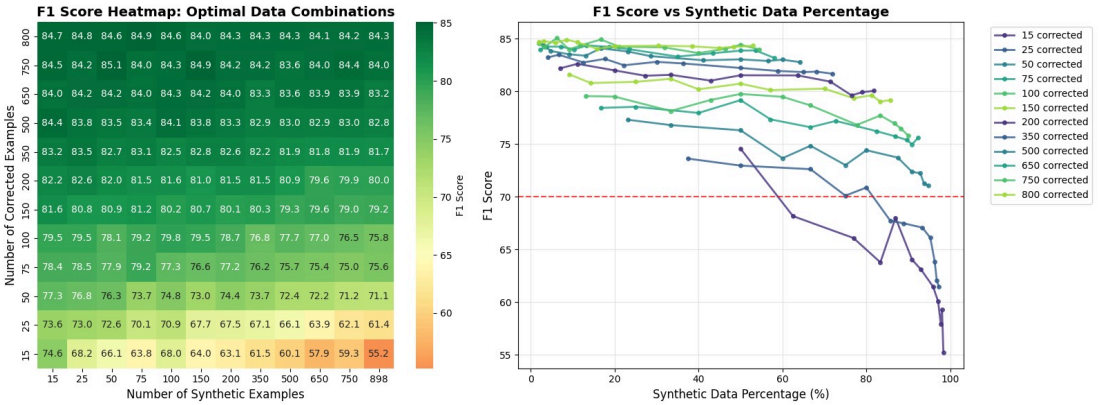

Fig 5.2.1 : Gliner finetuned model performance on ground truth data mixed with synthetic generated data on Min Active strategy

The synthetic examples were then mixed with different numbers of manually corrected examples from the original dataset and used to fine tune the span based model. Corrected examples denote sentences with human verified labels and therefore represent ground truth. The left panel of the figure summarises the results as a heatmap. The vertical axis shows the number of corrected examples, the horizontal axis shows the number of synthetic examples, and each cell reports the resulting F1 score on the test set. The heatmap reveals that for a fixed number of corrected examples, adding a small or moderate amount of synthetic data can maintain a high F1 score, but heavy reliance on synthetic data with very few corrected examples leads to a pronounced decline in performance. This effect is strongest in the rows with very few corrected examples, where increasing the number of synthetic sentences steadily pushes the F1 score downward.

The right panel presents the same information in terms of the percentage of synthetic data in the training set. Each curve corresponds to a fixed number of corrected examples, while the horizontal axis represents the share of synthetic data. Across almost all curves the F1 score decreases as the synthetic percentage grows. When only a small number of corrected examples is available, moving towards a training set that is dominated by synthetic data causes a sharp reduction in F1, often below the level achieved with a more balanced mix. When several hundred corrected examples are present, the curves are flatter, but the best results are still obtained when synthetic data remains a complement rather than the majority of the training set.

Overall, these experiments indicate that fully synthetic data generated by Gemma 3 12B cannot replace a substantial pool of human annotated examples for this task. Synthetic data can be helpful when used in moderation together with ground truth, but a high proportion of synthetic text and labels tends to move the model away from the true data distribution and leads to lower F1 scores, especially in low supervision regimes.

Chapter 6. Limitations

6.1 Complex NER extraction , relation extraction , question answering

The work in this thesis focuses on a relatively clean real world dataset with well defined entity types and annotations. All experiments are carried out on sentences that contain non nested and non overlapping entities, and the task is restricted to classical named entity recognition. This setting is representative of many production systems, but it is still simpler than many information extraction problems encountered in practice.

In many real applications the structure of the information is more complex. Text may contain nested entities where one mention is embedded inside another, overlapping entities where the same token belongs to several mentions, or discontinuous entities that span non contiguous segments. Beyond entity recognition, downstream tasks often require relation extraction between entities, document or passage level classification and question answering about entities and their attributes. In such cases the output space includes not only entity spans but also links, labels and answers, and the annotation schemes are less straightforward than in the dataset used here.

Recent work on span based models such as GLiNER has started to extend this paradigm beyond simple entity tagging. New variants target relation extraction, document classification and question answering, and expose interfaces that can in principle be fine tuned with the same ideas used in this thesis, namely active learning, LoRA adapters and mixtures of ground truth and synthetic labels. However, these models and their corresponding datasets are still relatively new, and there is limited understanding of how the data was prepared, what assumptions were made about entity and relation structure and how robust the models are across domains.

A natural next step is therefore to study how the methods developed in this thesis transfer to these more complex tasks. This will require careful work on data preparation and annotation guidelines, for example deciding how to represent nested and overlapping entities, how to encode relations and answers in a span based format and how to design prompts that elicit high quality synthetic labels for these richer structures. It will also require reevaluating active learning strategies and mixing ratios when the supervision signal involves both entities and relations or question answer pairs.

Ultimately, many realistic information extraction systems will need to combine several techniques in a coordinated pipeline. Span based models can provide flexible entity and relation candidates, large language models can contribute reasoning and natural language understanding, and multiple rule based components or knowledge bases can enforce domain specific constraints, validate outputs and correct systematic errors. The results of this thesis provide a first step by showing that careful fine tuning and data selection already deliver strong gains for relatively simple NER, but extending these ideas to complex extraction, relation modelling and question answering in combination with richer rule based systems remains an important direction for future research.

6.2 Multilingual limitations and opportunities for GLiNER

Most current GLiNER models are trained only on English data. A first multilingual variant already exists, but it is still early and its behaviour across languages and tasks is not yet well understood. This means that the core architecture is available, but its potential beyond English has only begun to be explored.

6.3 Addressing Catastrophic Forgetting

Catastrophic forgetting is a well known problem in neural networks where a model that is fine tuned on new data loses some of its earlier abilities. For span based NER this means that a model adapted to a narrow domain can become very good at the new entities but much worse at recognising the general entity types it learned during pre training. In production settings this is undesirable, because the same system often needs to handle both common queries and highly specialised documents.

One classical way to reduce catastrophic forgetting is to mix the new domain data with a small sample of the original training data during fine tuning. In practice this means building a combined training set that contains both domain specific sentences and sentences from the original corpus, and training on both at the same time. Prior work on GLiNER suggests using more original data than domain data, for example ratios around two to five original examples for each domain example, so that gradient updates must satisfy both types of supervision. This strategy helps the model preserve its general skills while still adapting to the new domain, but it increases training cost and requires careful curation of the replay data.

The approach taken in this thesis relies instead on LoRA adapters, which already provide a strong defence against catastrophic forgetting. The base GLiNER model remains frozen and only a small set of additional parameters is trained for each task. The original weights, which encode the broad NER knowledge acquired during pre training, are never modified. Domain specific behaviour is captured entirely in the adapters. As a result the risk of overwriting the base capabilities is greatly reduced, and the same base model can support many different adapters without repeatedly retraining or merging parameters.

This architecture also enables practical deployment strategies that further mitigate forgetting. Separate adapters can be maintained for different domains or customer applications, while a shared base adapter or the bare base model handles general traffic. A routing layer can choose the appropriate adapter based on metadata or simple heuristics and fall back to the general adapter when no specialised one is available. In this way the system preserves broad coverage, provides strong domain performance where adapters exist and avoids the cumulative degradation that would arise from repeatedly fine tuning the full model on ever more specialised datasets.

Chapter 7. Conclusions and Recommendations

7.1 Summary of Key Findings

This thesis investigated whether a compact span based model, tuned with LoRA and guided by active learning and synthetic supervision, can provide a realistic alternative

to directly using large language models for named entity recognition on the MIT Movies dataset.

The first finding is that LoRA based fine tuning of GLiNER can match full fine tuning performance while updating only a small part of the model. With a LoRA rank of 16 and a scaling factor of 32 applied to an all layers configuration, the model reached an F1 score of around 86.5 on the test set while training only about four to seven percent of the total parameters. The untuned base model stayed below 50 percent F1, while Gemma 3 12B reached about 69 percent, Qwen3 235B about 75 percent and Mistral 7B about 64 percent. At the same time, the experiments showed that LoRA must be applied carefully. Different GLiNER variants expose different internal module names and architectures, so target modules that work well for one model may be inappropriate for another and can silently lead to under training or unstable behaviour. Training runs should therefore be monitored for anomalies such as diverging loss or sudden drops in validation F1, and standard fine tuning best practices should be followed. Among all hyperparameters, the main learning rate and the secondary learning rate for non LoRA parameters proved to be the most sensitive, and small changes in these values had a larger impact on final performance than changes in other settings.

The second key result is that span based active learning strategies significantly improve data efficiency. By ranking training sentences using confidence measures such as least confidence, MNLP or MSE and always fine tuning on the most uncertain examples first, the model reached around 80 percent F1 with only a few hundred labelled sentences and climbed into the mid eighties once the budget reached roughly 1500 to 2000 examples. Across most subset sizes, these strategies consistently outperformed random sampling by one to two F1 points, showing that an organisation can reach its target quality with fewer annotations if examples are selected in a principled way.

The third major finding concerns the role of large language models as sources of supervision. Synthetic labels generated by models such as Gemma 3 12B and Qwen3 235B are valuable when mixed with human annotations but are not sufficient on their own. Training only on synthetic labels for the hardest examples produced F1 scores that saturated around the high sixties, close to the performance of the label generating model itself. In contrast, training on the same number of ground truth labels pushed the span model into the mid eighties. Mixed regimes showed that using roughly 50 to 75 percent ground truth and the rest synthetic labels already delivered F1 scores above 80 and often within a small margin of the fully supervised model once enough examples were available. Experiments with fully synthetic text and labels confirmed that large amounts of synthetic data tend to hurt performance, especially when the number of corrected human examples is small.

Taken together, these results show that a span based GLiNER model with carefully designed LoRA adapters, span aware active learning and a controlled use of synthetic supervision can deliver strong named entity recognition performance while keeping both compute and annotation costs under control. The exact configuration can then be tailored to the available data, the quality requirements and the budget of a specific application, rather than relying exclusively on very large language models or full model fine tuning.

7.2 Metric Selection and Evaluation Guidelines

In real information extraction systems, evaluation is not only about a single accuracy number. Predictions are often consumed by downstream applications or humans, and errors can have different costs depending on the business context. This thesis therefore treats metric selection and threshold choice as design decisions that must be aligned with the intended use case rather than as fixed properties of the model.

When ground truth labels are available, the primary metrics remain F1, precision and recall computed on entity spans. The experiments in this work consistently use span level F1 as the main indicator of quality, with precision and recall reported to understand the underlying trade off. Threshold sweeps for the span model show that lower confidence thresholds favour recall at the cost of precision, whereas higher thresholds reverse this balance. For example, the fine tuned GLiNER model moves from recall near ninety percent and lower precision at a threshold of zero point one to a more balanced precision and recall in the mid to high eighties at a threshold of zero point five. In practice, applications that must avoid missed entities, such as safety monitoring or medical alerting, may accept a lower precision regime, while applications that require very few false positives, such as legal contract review, should operate at a higher threshold even if recall drops slightly.

In settings where no ground truth is available, evaluation has to rely on indirect signals. Model confidence is the most important of these. Confidence based curves in this thesis, such as F1 against the share of worst confidence examples, illustrate how predictions behave as examples become more difficult. In production, similar analyses can be used to define operating bands where high confidence predictions are accepted automatically, medium confidence predictions are sent to human review and very low confidence predictions are flagged for further investigation or ignored. Over time, organisations can build small audited datasets from these reviewed cases to reintroduce direct F1, precision and recall measurement.

More complex extraction tasks will require richer metrics. The experiments in this thesis assume flat NER, with non nested and non overlapping entities, so standard span level F1 is appropriate. Once nested entities, relations or question answering are involved, evaluation will need to consider structured outputs. This may include separate metrics for entity detection, relation accuracy and answer correctness, or task specific measures that combine them. For span based models such as GLiNER, it is also important to be explicit about whether evaluation is carried out in flat or structured mode, since configuration flags such as flat NER determine which predictions are produced and how they are scored.

Finally, evaluation should always be interpreted together with cost related quantities. The active learning experiments in this thesis report F1 as a function of the number of labelled sentences, making the annotation budget an explicit part of the metric. The synthetic mixing experiments report F1 as a function of the proportion of synthetic labels, which reflects the trade off between annotation cost and quality. Similar curves can be constructed for inference cost by plotting F1 against the number of large language model calls or GPU time. For practitioners, the recommended guideline is therefore to report at least three elements for each configuration: span level F1 with its

associated precision and recall at the chosen threshold, the number and type of labelled examples used, and the main cost drivers such as annotation effort and model inference budget.

Chapter 8. Future work

8.1 Latest developments in Gliner family

The GLiNER model family continues to evolve with new variants that expand capabilities beyond standard named entity recognition. GLiNER2[16] introduces schema-driven interfaces that provide more flexible entity type specification and improved zero-shot generalization through enhanced entity type conditioning mechanisms. Preliminary evaluations suggest that GLiNER2 may offer superior performance on specialized domains or rare entity types compared to the original GLiNER architecture.

GLiNER multitask[15] extends the framework to handle diverse information extraction tasks including relation extraction, event detection, and slot filling within a unified architecture. This generalization enables organizations to deploy single models capable of addressing multiple extraction requirements rather than maintaining separate specialized systems. The parameter efficiency of LoRA fine-tuning combined with multi-task architectures may enable particularly cost-effective information extraction solutions where single adapters provide comprehensive extraction capabilities.

Future research should systematically evaluate these advanced variants using the mixing strategies and active learning approaches established in this work. The investigation of optimal training procedures, synthetic label generation methods, and evaluation frameworks for multi-task scenarios represents important next steps. The potential for knowledge transfer across related extraction tasks within multi-task frameworks may provide additional benefits beyond those observed for single-task scenarios.
The development of specialized GLiNER variants optimized for particular challenges such as nested entity recognition, discontinuous entity handling, or crosslingual transfer represents another valuable research direction. While the general GLiNER architecture demonstrates impressive flexibility, purpose-designed variants may achieve superior performance or efficiency for specialized applications. The community should continue exploring architectural innovations that push the boundaries of what lightweight models can accomplish.

8.2 Agentic AI and Integrated Frameworks

Recent work on agentic AI shows that language models are most powerful when they act as controllers of tools rather than as monolithic end to end systems. In this view, an agent plans a sequence of steps, calls specialised tools such as search, databases or extraction models, and then reasons over their outputs. Small and medium sized models can perform well in this setting because they delegate heavy computation to dedicated components.

GLiNER fits naturally into this trend as a specialised information extraction tool that an agent can call when structured entities are needed. Instead of asking a large language model to both read a document and reason about it in a single prompt, an agent can first invoke GLiNER to obtain reliable entity spans and types, and then pass these structured results to a reasoning model for further tasks such as summarisation, recommendation or decision support. This division of labour reduces token usage for the general model and makes the extraction behaviour easier to test and monitor.

In a practical system, GLiNER would sit inside an orchestration framework alongside retrieval, rule based validation and knowledge base access. A controller model could choose when to use GLiNER, which label schema to apply and how to interpret confidence scores. Low confidence extractions could trigger additional steps such as retrieving similar examples, calling a larger model for cross checking or asking a human to review. High confidence extractions could flow directly into downstream business logic. For multilingual or multi domain deployments, different GLiNER variants and adapters could be loaded dynamically based on document language, domain tags or user configuration.

Designing such integrated frameworks raises several research questions. These include how to route between GLiNER and general purpose models, how to combine extraction confidence with agent level uncertainty estimates, and how to log and replay complex extraction workflows for auditing. Extending the methods in this thesis to agentic settings would involve not only fine tuning GLiNER but also defining standard interfaces, schemas and monitoring practices so that extraction components can be composed safely with other tools. This direction aligns well with current trends in applied AI and offers a promising path toward robust, explainable and cost efficient information extraction systems.

8.4 Closing Perspective

This research demonstrates that the combination of lightweight GLiNER architectures, parameter-efficient LoRA fine-tuning, and strategic mixing of synthetic and ground truth labels enables practical named entity recognition systems that balance performance, efficiency, and cost. The findings establish that organizations need not choose between state-of-the-art accuracy and production feasibility, but can instead leverage knowledge distillation approaches that capture the benefits of large language models while maintaining the operational characteristics essential for enterprise deployment.

The path forward involves continued refinement of these techniques, extension to more complex information extraction tasks, integration with emerging agentic AI frameworks, and development of comprehensive production-grade implementations. As language models continue advancing and architectural innovations emerge, the opportunity exists to create information extraction systems that approach human-level performance while maintaining the efficiency, scalability, and reliability required for widespread adoption.

The techniques and insights presented in this work provide a foundation for organizations seeking to implement production information extraction systems and researchers investigating knowledge distillation, active learning, or efficient model deployment. The continued evolution of this research direction promises to

democratize access to sophisticated information extraction capabilities, enabling organizations of all sizes to derive value from unstructured text data without the prohibitive costs associated with large language model deployment

Chapter 9. References

[1] "Experiencing decreased performance with ChatGPT-4 | Hacker News." Accessed: Nov. 22, 2025. [Online]. Available: https://news.ycombinator.com/item?id=36633995

[2] N. A. Chinchor and B. Sundheim, "Message Understanding Conference (MUC) Tests of Discourse Processing".

[3] I. Keraghel, S. Morbieu, and M. Nadif, "Recent Advances in Named Entity Recognition: A Comprehensive Survey and Comparative Study," Dec. 20, 2024, *arXiv*: arXiv:2401.10825. doi: 10.48550/arXiv.2401.10825.

[4] "filtered semi markov crf." Accessed: Oct. 26, 2025. [Online]. Available: https://openreview.net/pdf?id=vNrmEgfGIg3

[5] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF Models for Sequence Tagging," Aug. 09, 2015, *arXiv*: arXiv:1508.01991. doi: 10.48550/arXiv.1508.01991.

[6] "Introducing LangExtract: A Gemini powered information extraction library-Google Developers Blog." Accessed: Sept. 13, 2025. [Online]. Available: https://developers.googleblog.com/en/introducing-langextract-a-gemini-powered-information-extraction-library/

[7] W. Zhou, S. Zhang, Y. Gu, M. Chen, and H. Poon, "UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition," Jan. 19, 2024, *arXiv*: arXiv:2308.03279. doi: 10.48550/arXiv.2308.03279.

[8] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer," Nov. 14, 2023, *arXiv*: arXiv:2311.08526. doi: 10.48550/arXiv.2311.08526.

[9] P. Belcak *et al.*, "Small Language Models are the Future of Agentic AI," June 02, 2025, *arXiv*: arXiv:2506.02153. doi: 10.48550/arXiv.2506.02153.

[10] K. Engineering, "GLiNER-Decoder: You extract entities only once," Medium. Accessed: Sept. 13, 2025. [Online]. Available: https://blog.knowledgator.com/gliner-decoder-you-extract-entities-only-once-1478e8d4a545

[11] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders," Aug. 21, 2024, *arXiv*: arXiv:2404.05961. doi: 10.48550/arXiv.2404.05961.

[12] I. Stepanov, M. Shtopko, D. Vodianytskyi, O. Lukashov, A. Yavorskyi, and M. Yaroshenko, "GLiClass: Generalist Lightweight Model for Sequence Classification Tasks," Aug. 11, 2025, *arXiv*: arXiv:2508.07662. doi: 10.48550/arXiv.2508.07662.

[13] R. Armingaud and R. Besançon, "GLiDRE: Generalist Lightweight model for Document-level Relation Extraction," Aug. 01, 2025, *arXiv*: arXiv:2508.00757. doi: 10.48550/arXiv.2508.00757.

[14] J. Boylan, C. Hokamp, and D. G. Ghalandari, "GLiREL -- Generalist Model for Zero-Shot Relation Extraction," Jan. 06, 2025, *arXiv*: arXiv:2501.03172. doi: 10.48550/arXiv.2501.03172.

[15] I. Stepanov and M. Shtopko, “GLiNER multi-task: Generalist Lightweight Model for Various Information Extraction Tasks,” Aug. 01, 2024, *arXiv*: arXiv:2406.12925. doi: 10.48550/arXiv.2406.12925.
[16] U. Zaratiana, G. Pasternak, O. Boyd, G. Hurn-Maloney, and A. Lewis, “GLiNER2: An Efficient Multi-Task Information Extraction System with Schema-Driven Interface,” July 24, 2025, *arXiv*: arXiv:2507.18546. doi: 10.48550/arXiv.2507.18546.
[17] “LoRA Without Regret,” Thinking Machines Lab. Accessed: Oct. 19, 2025. [Online]. Available: https://thinkingmachines.ai/blog/lora/
[18] A. Siddhant and Z. C. Lipton, “Deep Bayesian Active Learning for Natural Language Processing: Results of a Large-Scale Empirical Study,” Sept. 24, 2018, *arXiv*: arXiv:1808.05697. doi: 10.48550/arXiv.1808.05697.
[19] Y. Shen, H. Yun, Z. Lipton, Y. Kronrod, and A. Anandkumar, “Deep Active Learning for Named Entity Recognition,” in *Proceedings of the 2nd Workshop on Representation Learning for NLP*, P. Blunsom, A. Bordes, K. Cho, S. Cohen, C. Dyer, E. Grefenstette, K. M. Hermann, L. Rimell, J. Weston, and S. Yih, Eds., Vancouver, Canada: Association for Computational Linguistics, Aug. 2017, pp. 252–256. doi: 10.18653/v1/W17-2630.
[20] Y. Heng *et al.*, “ProgGen: Generating Named Entity Recognition Datasets Step-by-step with Self-Reflexive Large Language Models,” Mar. 17, 2024, *arXiv*: arXiv:2403.11103. doi: 10.48550/arXiv.2403.11103.
[21] K. M. Yoo, D. Park, J. Kang, S.-W. Lee, and W. Park, “GPT3Mix: Leveraging Large-scale Language Models for Text Augmentation,” Nov. 18, 2021, *arXiv*: arXiv:2104.08826. doi: 10.48550/arXiv.2104.08826.
[22] N. D. L. Fuente, O. Sainz, I. García-Ferrero, and E. Agirre, “GuideX: Guided Synthetic Data Generation for Zero-Shot Information Extraction,” May 31, 2025, *arXiv*: arXiv:2506.00649. doi: 10.48550/arXiv.2506.00649.
[23] G. Kamath and S. Vajjala, “Does Synthetic Data Help Named Entity Recognition for Low-Resource Languages?,” May 22, 2025, *arXiv*: arXiv:2505.16814. doi: 10.48550/arXiv.2505.16814.
[24] “LLM Leaderboard - Comparison of over 100 AI models from OpenAI, Google, DeepSeek & others.” Accessed: Nov. 25, 2025. [Online]. Available: https://artificialanalysis.ai/leaderboards/models
[25] “Vertex AI pricing,” Google Cloud. Accessed: Nov. 25, 2025. [Online]. Available: https://cloud.google.com/vertex-ai/pricing
[26] “Pricing.” Accessed: Nov. 25, 2025. [Online]. Available: https://openai.com/api/pricing/
[27] “Pricing,” Amazon Web Services, Inc. Accessed: Nov. 25, 2025. [Online]. Available: https://aws.amazon.com/ec2/pricing/
[28] “(PDF) Model Monitoring, Data Drift Detection, and Efficient Model Retraining: A Review,” ResearchGate. Accessed: Nov. 25, 2025. [Online]. Available: https://www.researchgate.net/publication/395703466_Model_Monitoring_Data_Drift_Detection_and_Efficient_Model_Retraining_A_Review
[29] L. Faubel and K. Schmid, “MLOps: A Multiple Case Study in Industry 4.0,” July 12, 2024, *arXiv*: arXiv:2407.09107. doi: 10.48550/arXiv.2407.09107.